HIDDEN-STATE GATING FOR MULTI-HOP RETRIEVAL

by

E ERTAI

(B.Sc. in Artificial Intelligence, Beijing Institute of Technology)

A DISSERTATION SUBMITTED FOR THE DEGREE OF

MASTER OF COMPUTING

in

ARTIFICIAL INTELLIGENCE

in

SCHOOL OF COMPUTING

of the

NATIONAL UNIVERSITY OF SINGAPORE

2026

Supervisor:
Professor TUNG Kum Hoe Anthony

Declaration

I hereby declare that this dissertation is my original work and it has been written by me in its entirety. I have duly acknowledged all the sources of information which have been used in the dissertation.

This dissertation has also not been submitted for any degree in any university previously.

E Ertai

1 Oct 2026

Acknowledgments

[**TODO:** Acknowledgments.]

Contents

Acknowledgments **i**

Abstract **v**

List of Figures **vi**

List of Tables **viii**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Problem Statement 2

1.3 Contributions 5

1.4 Dissertation Synopsis 6

2 Literature Review **7**

2.1 Multi-Hop Question Answering and Retrieval 7

2.2 Adaptive Retrieval and Evidence Assessment 10

2.3 Reading Reasoning Signals from Hidden States 11

3 Methodology **14**

3.1 Problem Formulation 14

3.2 Gate-Guided Retrieval 15

3.2.1 Candidate Scoring and Reranking 16

3.2.2 Chain Update and Answer Generation 18

3.3 Hidden-State Gate 19

3.3.1 Counterfactual Trace Construction 20

3.3.2 Gate Training Data and Balancing 22

3.3.3 Hidden-State Features and Gate Scoring 24

3.4 Question Decomposer . . . 26
3.4.1 Decomposition Supervision and Target Format . . . 27
3.4.2 Decomposer Training and Inference . . . 27

4 Experiments and Results **29**
4.1 Experimental Setup . . . 29
4.1.1 Datasets . . . 29
4.1.2 Implementation Details . . . 30
4.1.3 Evaluation Metrics . . . 31
4.2 Component Evaluation . . . 32
4.2.1 Hidden-State Structure . . . 32
4.2.2 Gate . . . 34
4.2.3 Decomposer . . . 35
4.3 Main Results . . . 36
4.4 Ablation Studies . . . 38
4.4.1 Gate Features . . . 38
4.4.2 Layer Selection . . . 38
4.4.3 Fusion Weight . . . 39
4.4.4 Final-Answer Prompt . . . 40
4.4.5 Decomposer Training Data . . . 41
4.5 Robustness . . . 42
4.5.1 Alternative Retriever . . . 42
4.5.2 Open-Corpus Retrieval . . . 43
4.6 Comparison with Prior Systems . . . 44

5 Discussion **47**
5.1 What the Gate Detects . . . 47
5.2 Error Analysis . . . 47
5.3 Limitations . . . 47
5.4 Future Directions . . . 48

6 Conclusion **49**

Bibliography **50**

A Design Explorations **56**

B Prompts **57**

B.1 Answer of Each Hop 57

B.2 Final Answer 58

Abstract

Hidden-State Gating for Multi-Hop Retrieval

by

E Ertai

Master of Computing in Artificial Intelligence

National University of Singapore

[**TODO:** Abstract (written last, 300 words): problem, method, main results, analysis finding.]

List of Figures

1.1 Problem setup (illustrative example). (a) Error propagation along a reasoning chain. In hop 1, the candidates are ranked by retrieval similarity, and Passage 1 (red), which mentions *Inception* but not its director, is ranked above the correct Passage 2 (green). Its answer, "Hans Zimmer", fills the placeholder of the hop-2 sub-question, which then retrieves evidence about the wrong person, and the final answer is wrong. (b) The decision at each hop, shown for hop 1. The current sub-question and each candidate are appended to the prefix (at hop 1, only the question) and encoded by the frozen LLM; the gate maps each resulting hidden state to a risk score (values are illustrative). After reranking, Passage 2 is placed first and passed on to the following hops. 4

3.1 One hop of gate-guided retrieval. (1) Retrieval: the placeholders of the sub-question q_j are filled with the earlier answers, and the expanded sub-question $\tilde{q}_j$ is matched against the pool by the BGE encoder, giving the candidates $\mathcal{C}_j$ and their retrieval scores $s_j(p)$; for brevity, the retrieval instruction and the passage titles of Equation 3.1 are omitted. (2) Risk: for each candidate, the step "Step j: q_j Evidence: p" is appended to the prefix $x_{<j}$, with the placeholders of q_j left unfilled, and encoded by the frozen LLM; the gate g_j maps the state at the end of the candidate, $\mathbf{h}_j(p)$, and its difference from the state at the end of the prefix, $\mathbf{u}_j$, to a risk score $r_j(p)$. (3) Selection: the candidate with the highest $s_j(p) - \lambda r_j(p)$ is selected as e_j (values are illustrative). (4) Chain update: the LLM generates the answer a_j, which fills the placeholders of later sub-questions, and (q_j, e_j) is appended to the prefix for hop $j+1$. . . . 17

3.2 Counterfactual trace construction for a three-hop question. Left: for the substituted hop j^*, the sub-question is filled with the annotated answers of the earlier hops, the pool is ranked by the retriever, passages annotated as supporting any hop are skipped, and the three highest-ranked remaining passages of the top ten are used as replacements. Right: the gold chain (top) and the counterfactual chains for each $j^* \in \{1, 2, 3\}$; each counterfactual row stands for three chains, one per replacement. Annotated passages (e_j^*) are shown in green, substituted passages (p') in red, and hops after j^*, which are discarded, in gray. Each remaining hop yields one example, consisting of the prefix and the step of that hop, with the label y_j shown below it. In every example the prefix contains only annotated passages, and a substituted passage appears exactly in the examples with $y_j = 1$. 22

4.1 t-SNE projection of the layer-15 states of gold chains, one panel per chain length K, each with 1,000 training chains and all of their prefixes. Colors indicate the prefix index, from the question alone (1) to the full chain ($K + 1$). The axes of a t-SNE projection have no units. 33

4.2 Projection of development states onto the mean difference between substituted and gold evidence, for each chain length K and substituted hop. The direction is estimated on one half of 400 question pairs, and the histograms and the AUC in each title are computed on the other half. Blue: gold evidence; red: substituted evidence. 35

5.1 Gate score relative to the first error in the chain. 48

List of Tables

3.1 Notation used in this chapter. 16
3.2 The two trained components of the method. 20
3.3 Gate-training traces and resulting hop-level examples. The training split includes the paraphrased variants of three- and four-hop questions; only the balanced chains are used. Each chain contributes one example to every hop position it reaches, so the number of examples decreases with the hop position. 23
3.4 Texts read by the gate for the example of Figure 1.1. Each hop appends one step (in bold) to the prefix. The second sub-question keeps its placeholder, and the answer of the first hop does not appear. 24

4.1 Development sets used for evaluation. In 2WikiMultihopQA, the four-hop questions are exactly the bridge comparison questions (2,751); the remaining questions are compositional (5,236), comparison (3,040), and inference (1,549) questions. HotpotQA contains 5,918 bridge and 1,487 comparison questions. 30
4.2 Hyperparameters of the method and how they were set. 31
4.3 Accuracy of predicting the prefix index of gold development chains, with a linear classifier trained on training chains. "Token count only" uses the length of the prefix as its single feature. 33
4.4 The gate as a classifier on the MuSiQue development chains, at the threshold set on the training data. The AUC over all positions is 0.936 (precision 0.715, F1 0.791). 35
4.5 Proportion of development questions for which the decomposer predicts the annotated number of hops. 36

4.6 Main results on the development sets. The control differs from the method only in the selection rule ($\lambda = 0$, selection by retrieval score). The gate is trained on MuSiQue only. 37
4.7 Paired comparison of EM between the method and the control, in points. The middle columns count the questions answered correctly by only one of the two. 37
4.8 Gates trained on the transition only, the accumulated state only, and both, with all other settings fixed. 39
4.9 AUC of the gate on held-out training questions for each candidate layer (counting from zero). The development data were not used. 39
4.10 Results on 3,000 MuSiQue training questions for different values of λ (lenient chain@1). 40
4.11 EM with and without the instruction for comparison questions in the final-answer prompt. Only the final answer is regenerated; retrieval and the answers of individual hops are identical within each system. 41
4.12 The method with decomposers trained with and without the generated 2WikiMultihopQA decompositions. Recall@1 counts every annotated hop, with hops that are not predicted counted as misses. 42
4.13 Results with ColBERTv2 as the retriever (lenient chain@1). The gate is the same as in the main experiments and is not retrained. 43
4.14 Open-corpus results with k candidates per hop (lenient chain@1). The distractor column repeats the main result of the method for reference. . 43
4.15 Comparison with prior systems. Recall is order-invariant and passages are matched by title. Coverage is the fraction of annotated passages in the evidence finally used (mean over questions) and the fraction of questions for which all of them are included (full). 45
4.16 Average number of passages in the evidence finally used for answering, and the proportion of them that are annotated supporting passages (precision). The questions have 2.60, 2.44, and 2.00 supporting passages on average. 46

A.1 Design explorations that were not adopted. 56

Chapter 1

Introduction

1.1 Background and Motivation

Large language models (LLMs) store a substantial amount of world knowledge in their parameters, yet they remain unreliable on questions that depend on specific or long-tail facts. Retrieval-augmented generation (RAG) addresses this by conditioning the model on passages retrieved from an external corpus [1]. Because the generator's external evidence is limited to what the retriever returns, retrieval quality largely bounds the quality of the final answer. This dependence is most pronounced for multi-hop questions, which require composing facts that are spread across several documents [2, 3, 4]. Answering "Where was the director of *Inception* born?", for example, requires first identifying the director and then finding that person's birthplace, and no single passage is likely to contain both facts. A common strategy is to decompose such a question into a sequence of simpler sub-questions and to retrieve evidence for each of them in turn [5, 6, 7], filling the answer of one hop into the sub-question of the next: here, "Who directed *Inception*?" followed by "Where was [Answer 1] born?".

This sequential structure also makes decomposition-based retrieval fragile. Each hop depends on the answer produced by the hop before it, so a wrong passage need not remain a local mistake: its incorrect answer is substituted into the next sub-question, which can steer the next retrieval toward the wrong entity and carry the error into later hops. Errors at individual hops therefore accumulate along the chain, and the proportion of questions whose entire evidence chain is retrieved correctly is considerably lower than the per-hop retrieval accuracy, with the gap widening as chains grow longer. A complete and correct evidence chain is not strictly necessary

for a correct final answer, since the reader may recover from a missing fact or draw on its own knowledge, but it is the most direct route to one. For this reason, we evaluate retrieval both at the level of individual hops and at the level of whole chains.

Existing approaches address this problem along two lines. Adaptive and self-corrective retrieval methods decide during generation whether to retrieve and whether the retrieved evidence can be trusted [8, 9, 10, 11], but they rely on additional generation and retrieval rounds, separately trained evaluators, or a fine-tuned generator. Alternatively, stronger multi-hop retrievers can be trained directly [12], at a considerable training cost and with gains tied to a particular retrieval architecture. This motivates lighter alternatives that assess each hop's evidence without additional decoding, without retraining the retriever, and without modifying the generator.

A promising source for such a signal is the generator itself. Recent work shows that the hidden states of an LLM reflect whether its own reasoning is on track: in step-by-step reasoning, a linear classifier over hidden states at step boundaries can predict whether the final answer will be correct before it is produced [13]. That study, however, examines mathematical and multiple-choice reasoning and names multi-hop question answering as an open setting. Multi-hop retrieval differs in an important respect: each step incorporates an external passage chosen by a retriever, so errors can originate in the selection of evidence, in addition to the model's own reasoning. This leads to the central question of this dissertation: *can the hidden states of the generator, read at each hop, reveal whether a retrieved passage will lead the reasoning chain astray, and can this signal be used to guide retrieval?*

1.2 Problem Statement

We study multi-hop question answering in which each question comes with a pool of candidate passages, only some of which contain the facts needed to answer it. The question is first decomposed into a sequence of sub-questions, where later sub-questions contain placeholders that refer to the answers of earlier ones. The system then proceeds hop by hop. At each hop, the current sub-question is completed with the answers obtained so far, a retriever ranks the candidate passages by their similarity to it, one passage is selected as the evidence for that hop, and a short

answer is generated from the selected passage. After the last hop, a reader produces the final answer from the full chain of sub-questions, evidence, and intermediate answers. Fig. 1.1(a) illustrates this process. The task we address is the selection made at each hop: given the question, the steps completed so far, the current sub-question, and the retrieved candidates, select the passage that keeps the whole reasoning chain correct.

This selection is difficult for two reasons. First, a wrong passage is often hard to recognize at the hop where it is selected. The passages that mislead a chain are rarely random noise; they typically share entities and wording with the sub-question, and therefore receive high retrieval scores. In Fig. 1.1(a), the soundtrack passage mentions *Inception* and a person's name, and is ranked above the passage that actually names the director. Second, the consequences of a wrong selection only become visible later. Once its answer is substituted into the next sub-question, subsequent hops may retrieve evidence that is correct for the question they are asked, as the passage about Hans Zimmer is in hop 2, while the chain as a whole has already failed. A decision is therefore most useful if it is made at the hop itself, before the error propagates. We further require that it be made without modifying the generator or retraining the retriever, and without any additional decoding.

Our approach is to let the generator assess each candidate. As illustrated in Fig. 1.1(b), the current sub-question and a candidate are appended to the prefix of the chain, which consists of the question and the steps completed so far, and the result is encoded by the frozen generator. A lightweight gate then maps the resulting hidden state to a risk score, the candidates are reranked by their retrieval scores adjusted by this risk, and the top-ranked passage is selected. Scoring a candidate requires a single forward pass over this text, with no tokens generated. Training the gate requires labels that indicate which hop of a chain has gone wrong, which natural model errors do not provide reliably. We therefore construct counterfactual reasoning chains in which exactly one hop is given a highly ranked passage other than its annotated supporting passage, using a dataset that explicitly annotates the supporting passage of every hop. The label of each hop is thus determined by construction, and the gate is trained to recognize the substituted hop. The method is described in detail in Chapter 3.

We evaluate the approach on three multi-hop question answering benchmarks,

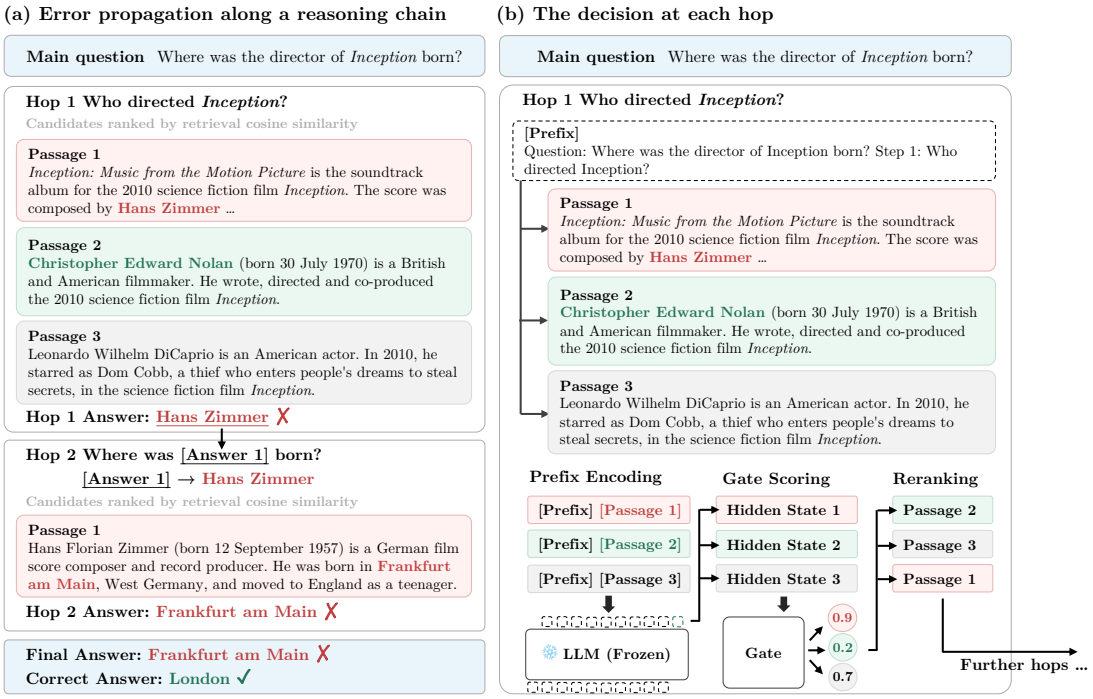


Figure 1.1: Problem setup (illustrative example). (a) Error propagation along a reasoning chain. In hop 1, the candidates are ranked by retrieval similarity, and Passage 1 (red), which mentions *Inception* but not its director, is ranked above the correct Passage 2 (green). Its answer, "Hans Zimmer", fills the placeholder of the hop-2 sub-question, which then retrieves evidence about the wrong person, and the final answer is wrong. (b) The decision at each hop, shown for hop 1. The current sub-question and each candidate are appended to the prefix (at hop 1, only the question) and encoded by the frozen LLM; the gate maps each resulting hidden state to a risk score (values are illustrative). After reranking, Passage 2 is placed first and passed on to the following hops.

MuSiQue [4], 2WikiMultihopQA [3], and HotpotQA [2]. The gate is trained only on MuSiQue, the only one of the three that explicitly annotates the sub-question and supporting passage of every hop, so the results on the other two benchmarks measure how well the gate transfers to datasets it has not seen; none of the components we train uses any HotpotQA data. We report performance at three levels: retrieval accuracy at each hop, whether the entire evidence chain of a question is retrieved correctly, and the quality of the final answer.

1.3 Contributions

The main contributions of this work are as follows.

First, we implement a lightweight gate that reads the hidden states of a frozen generator to score, at each hop, the candidate passages of decomposition-based multi-hop retrieval, and we use this score to rerank the candidates. The gate combines two signals from the generator: the accumulated state of the reasoning prefix after the candidate is added, and the change that the candidate induces in this state. It consists of a small linear classifier for each hop position and is trained on counterfactual reasoning chains in which exactly one hop is given a highly ranked passage other than its annotated supporting passage, so that every hop receives a label by construction. The generator and the retriever are left unchanged, and scoring a candidate requires one forward pass without decoding.

Second, we show through experiments on three multi-hop question answering benchmarks that the gate consistently improves both the proportion of questions whose entire evidence chain is retrieved correctly and the accuracy of the final answer. These gains carry over to the two benchmarks on which the gate was not trained, and to a retriever with a different scoring mechanism. Compared under a unified protocol with two recent multi-hop retrieval systems, the approach yields higher answer accuracy on two of the benchmarks and comparable accuracy on the third, without fine-tuning the generator or the retriever.

Finally, we analyze what the gate has learned, using reasoning chains that contain multiple errors at controlled positions. The analysis indicates that the gate's score behaves more like a detector of whether the reasoning prefix has been contaminated by an earlier error than like a locator of the hop at which the error occurred, and that within the gate the accumulated state carries almost all of the discrimination, while the per-step change, although informative on its own, adds little once the accumulated state is available. This behavior is consistent with the gains concentrating at the level of whole chains, and it also delineates the limits of the approach: once an error has occurred, later correct candidates tend to be penalized as well, and the gate cannot indicate which hop to revisit.

1.4 Dissertation Synopsis

The rest of this dissertation is organized as follows. Chapter 2 reviews prior work on multi-hop question answering and retrieval, adaptive and self-corrective retrieval, and the use of hidden states to read signals about a model's reasoning, situating the present work in relation to each. Chapter 3 formalizes the problem and presents the methodology, covering how a hidden-state risk score is used to select evidence, how the gate that produces this score is trained, and how the question decomposer is trained. Chapter 4 describes the experimental setup and reports the main results, ablation studies, robustness experiments, and a comparison with prior systems. Chapter 5 examines what the gate detects, analyzes the remaining errors, and discusses limitations and future directions. Chapter 6 concludes the dissertation.

Chapter 2

Literature Review

This chapter reviews the three bodies of literature on which the present work builds. Section 2.1 reviews multi-hop question answering and the retrieval methods developed for it, and identifies the decomposition-based framework that this dissertation adopts together with its central weakness. Section 2.2 surveys methods that decide during generation whether retrieval is needed and methods that assess retrieved evidence, including those that read the model's internal representations. Section 2.3 reviews work on reading information from the hidden states of language models, culminating in the reasoning-trajectory study that directly motivates this dissertation.

2.1 Multi-Hop Question Answering and Retrieval

Multi-hop question answering requires a system to combine facts that are spread across several documents. HotpotQA [2] established the task at scale with 113k Wikipedia-based questions that require reasoning over two paragraphs, either by following a bridge entity from one paragraph to the next or by comparing two entities. Each question is annotated with sentence-level supporting facts. Min et al. [14] later showed that a single-hop model reaches a performance comparable to that of multi-hop models on HotpotQA, because many questions target a specific entity type or rely on redundant facts and can therefore be answered without combining evidence. Subsequent datasets were designed with this concern in mind. 2WikiMultihopQA [3] generates questions from Wikipedia and Wikidata with templates and logical rules, covering compositional, comparison, inference, and bridge-comparison questions, and adds evidence information in the form of a set of knowledge-base triples that describes the reasoning path. MuSiQue [4] composes questions bottom-

up from pairs of connected single-hop questions and applies filters targeting connected reasoning, yielding 25k questions of two to four hops that are considerably harder to answer through shortcuts.

The three datasets differ in how finely the reasoning is annotated, which matters for the present work. HotpotQA marks the supporting sentences of each question, but not the order in which the two supporting paragraphs are used. 2WikiMultihopQA adds the evidence triples, but expresses each step as a knowledge-base fact rather than as a sub-question, provides no decomposition of the question, and does not state which paragraph supports which step. MuSiQue annotates each question with its decomposition: for every hop, a single-hop question, its answer, and the index of the paragraph that supports it. These hop questions are written either in natural language or in a compact form that pairs an entity with a relation, and refer to the answers of earlier hops through placeholders such as "#1". MuSiQue is therefore the only one of the three that directly provides per-hop supervision, which determines how the training data of the method is constructed in Chapter 3.

A single round of retrieval, as in standard retrieval-augmented generation [1], is poorly suited to such questions. The evidence for a later hop often shares little vocabulary with the original question, because it is about an entity that is only revealed by an earlier hop: a passage stating where a director was born may be difficult to retrieve directly from a question that never names the director. Multi-hop retrieval therefore proceeds in steps, with each step conditioned on what earlier steps have found. Existing methods differ mainly in how this conditioning is realized.

One line of work conditions retrieval directly on the evidence retrieved so far, without decomposing the question. MDR [15] trains a dense retriever that encodes the question together with the previously retrieved passages as the query for the next hop. Baleen [16] condenses the passages retrieved at each hop into a compact context, uses a late-interaction retriever that lets different parts of the query match different passages, and lets the retriever itself infer the order of the hops during training. Beam Retrieval [17] trains the retriever end to end across all hops and keeps several partial chains of passages at each step, which reduces the risk of discarding a relevant passage early; it reports results on MuSiQue, 2WikiMultihopQA, and HotpotQA. IRCoT [7] interleaves retrieval with chain-of-thought generation, using each newly generated reasoning sentence as the query for the next retrieval. Most

recently, GRITHopper [12] fine-tunes a 7B decoder model to perform decomposition-free multi-hop dense retrieval, concatenating the retrieved evidence with the query at each step, and achieves state-of-the-art retrieval performance. Methods in this line are effective, but most of them require training a retriever with multi-hop supervision, and their gains are tied to the retrieval architecture they train.

The other line of work first decomposes a multi-hop question into single-hop sub-questions and then retrieves and answers them in turn. DecompRC [5] decomposes questions into sub-questions, answers them with a single-hop reading comprehension model, and rescores alternative decompositions. Break [18] formalized question decomposition as an ordered list of natural-language steps and released the Break dataset, which pairs over 83,000 questions with their decompositions. With large language models, decomposition can be obtained through prompting: Self-Ask [6] has the model pose and answer its own follow-up questions, optionally answering them with a search engine. ChainRAG [19] identifies a failure specific to this setting, which the authors call lost-in-retrieval: sub-questions produced by decomposition often omit the key entity they depend on, so retrieval for them fails. It addresses the problem by handling the sub-questions in sequence, completing the missing key entities of each one, and retrieving relevant sentences from a sentence graph. Decomposition makes the reasoning process explicit and can be combined with any off-the-shelf retriever, which is why this dissertation adopts it.

The same explicitness, however, exposes the central weakness of the decomposition-based framework: each sub-question depends on the answers to earlier ones, so the passage selected at one hop shapes what is retrieved at the next. The lost-in-retrieval problem identified by ChainRAG is one manifestation of this dependency. Existing methods address it in different ways and at different costs. Beam Retrieval scores each candidate jointly with the question and the passages already selected, using an encoder trained with multi-hop supervision, and keeps several partial chains alive at the cost of scoring more combinations. ChainRAG, while expanding the sentence graph, asks an LLM whether the gathered sentences suffice to answer the sub-question, which adds LLM calls and, as we show in Chapter 4, produces a larger and noisier context. GRITHopper trains a stronger retriever at a considerable training cost. A related line of work judges retrieval during generation rather than at retrieval time, and we review it next.

2.2 Adaptive Retrieval and Evidence Assessment

A first group of methods decides when retrieval should take place. Adaptive-RAG [10] trains a small classifier to predict the complexity of a question and, based on this prediction, chooses between answering without retrieval, retrieving once, and retrieving iteratively. FLARE [8] generates a tentative next sentence and retrieves new evidence whenever that sentence contains low-confidence tokens, using the sentence itself as the query. DRAGIN [20] refines this idea by deciding when to retrieve from the uncertainty and importance of the tokens being generated, and by forming the query from the tokens the model attends to most. Along this progression, the signal that triggers retrieval moves from properties of the question to the model's own output distribution and attention. These methods, however, decide whether to retrieve more; they do not judge which of the passages already retrieved should be used.

A second group of methods judges the retrieved evidence itself. Self-RAG [9] fine-tunes the generator to emit special reflection tokens that indicate whether retrieval is needed, whether a retrieved passage is relevant, and whether the generated output is supported by it. CRAG [11] instead fine-tunes a separate lightweight evaluator, based on T5-large, that scores the retrieved documents for a query; depending on its confidence, the system keeps and refines the documents, discards them in favor of a web search, or combines both. Both methods can recognize unhelpful evidence, but they require either fine-tuning the generator or training and running an additional evaluator, and neither is designed specifically for the chain of dependent selections that arises in decomposition-based multi-hop retrieval.

A third group of methods assesses evidence by reading the internal representations of a language model. REAR [21] adds a rank head to the generator: a linear layer maps the last-token hidden state of the question and a document to a relevance score, and the head is optimized jointly with the generator, using relevance labels derived from a cross-encoder and binary annotations. RSRank [22] keeps the language model frozen and measures the representational shift that a document induces in the internal state of the query; lightweight projections, trained with document-level relevance labels, map this shift to calibrated relevance scores, at the cost of a single prefill pass per document. It is evaluated on HotpotQA, 2WikiMultihopQA, and

MuSiQue, but as a single-stage reranker rather than within an iterative retrieval process. Knowing Before Answering [23] trains a linear model on hidden activations and attention-derived features to classify, before any answer is generated, whether the retrieved context as a whole is sufficient, insufficient, or conflicting.

SeaKR [24] takes a different approach and estimates the model's uncertainty from its internal states and uses this estimate throughout an iterative retrieval process: to decide when to retrieve, to select among retrieved snippets the one that leaves the model least uncertain, and to choose a reasoning strategy for the final answer. Its uncertainty is computed by sampling multiple generations for the same context and measuring the consistency of their internal representations, which requires additional decoding for every candidate it evaluates.

Together, these methods show that a model's internal representations carry usable information about retrieved evidence. REAR, RSRank, and Knowing Before Answering, however, assess documents for a single query or a fixed context. In decomposition-based multi-hop retrieval, the evidence selected at one hop changes the sub-question and the context of the next, so what a representation reveals about a candidate may depend on what was selected before it. How hidden states behave when reasoning unfolds over such dependent steps is the subject of the next section.

2.3 Reading Reasoning Signals from Hidden States

A common way to ask what a neural network represents is to train a simple classifier, or probe, to predict a property of interest from the network's intermediate representations. Alain and Bengio [25] introduced linear classifier probes and observed that the linear separability of features tends to increase with depth. Belinkov [26] reviews the promises and shortcomings of this methodology and cautions that a probe that is too expressive may learn the task itself rather than reveal information already present in the representation, so that simple probes and controlled comparisons are needed to draw reliable conclusions.

A growing body of work shows that large language models carry information about whether a statement is true and whether their own answer is likely to be correct. Kadavath et al. [27] found that large models are well calibrated on multiple-choice and true/false questions when these are presented in a suitable format, and

that they can be trained to predict whether they know the answer to a question. Azaria and Mitchell [28] trained a feedforward classifier on hidden-layer activations to predict the truthfulness of statements, including statements the model itself generated. Burns et al. [29] showed that a direction separating true from false statements can be found in activation space without any labels, by requiring that a statement and its negation receive opposite truth values. Marks and Tegmark [30] provided evidence that, at sufficient scale, truth is represented linearly, with probes that transfer across datasets and interventions that flip the model's judgment. INSIDE [31] measures the consistency of the internal states of multiple sampled responses to detect hallucinations. These studies differ in their methods, ranging from linear directions to feedforward classifiers and sampling-based consistency measures, but together they show that correctness-related information can be recovered from hidden states. They examine isolated statements or single answers, however, rather than a reasoning process that unfolds over several steps.

Sun et al. [13] extend this line of work from statements to step-by-step reasoning, and their study is the direct starting point of this dissertation. Working with Llama 3.1 8B, primarily on the mathematical reasoning benchmarks GSM8K and MATH-500, they prompt the model to mark each reasoning step explicitly and extract the hidden state at the token immediately preceding each step marker, which summarizes the reasoning completed so far. They find that these states occupy step-specific regions of the representation space that become increasingly linearly separable in deeper layers, so that the sequence of states forms a structured trajectory.

They further find that the trajectories of correct and incorrect solutions are similar in the early steps and diverge in the late steps. A logistic regression over features of the late-step trajectory predicts whether the final answer will be correct before it is produced. Their most predictive features combine the state at the end of the reasoning with its change over the last step, and outperform both a baseline based on the number of steps and features derived from the output logits. Finally, they show that interventions applied only when this predictor anticipates a failure are more effective than the same interventions applied unconditionally. Their analysis extends to MMLU in transfer experiments, but the authors explicitly name multi-hop question answering as a setting in which it remains open whether similar

structure arises.

Beyond reading information out of hidden states, a related line of work modifies them to change the model's behavior. Inference-time intervention [32] shifts the activations of selected attention heads along directions associated with truthfulness and substantially improves the truthfulness of the model's answers, and representation engineering [33] develops a broader framework for reading and controlling high-level concepts through representations. Sun et al. likewise steer reasoning trajectories that deviate from those of correct solutions.

Returning to reading hidden states, recent work on multi-step agents points to a complication that arises once reasoning steps depend on one another. Kim et al. [34] observe that hidden-state probes for step correctness degrade when the prefix contains earlier errors: the probes track whether a step is consistent with the possibly erroneous prefix rather than whether it is correct, so a step that continues an earlier error can look correct and a step that repairs it can look wrong. They call this prefix contamination and correct the probe with attention-based features in order to obtain step-level rewards for reinforcement learning. Although their setting, scoring the generated actions of tool-using agents, differs from evidence selection in multi-hop retrieval, the phenomenon they describe can arise whenever hidden states are read along a chain of dependent steps.

Taken together, these studies show that hidden states encode whether reasoning is on track, both for isolated statements and for step-by-step reasoning, and that in multi-step settings this signal becomes entangled with the consistency of the prefix. In the studies reviewed in this section, however, what is judged is either a given statement or the model's own reasoning, whereas in multi-hop retrieval each step brings in an external passage chosen by a retriever.

Chapter 3

Methodology

This chapter presents the method studied in this dissertation. Section 3.1 formalizes decomposition-based multi-hop retrieval. Section 3.2 describes how the method selects the evidence of each hop by combining retrieval similarity with a risk score, and how the chain and the answers are produced. The two components that this selection relies on are then described: Section 3.3 presents the gate that assigns the risk score, and Section 3.4 presents the decomposer that produces the sub-questions.

3.1 Problem Formulation

The pipeline uses two models. A dense retriever, the BGE encoder bge-base-en-v1.5 [35], ranks passages. A single large language model, Llama-3.1-8B-Instruct [36], serves three purposes: it generates the answer of each hop, it produces the final answer, and its hidden states are used to assess candidate passages. We refer to it as the LLM throughout.

Let Q be a multi-hop question and $\mathcal{P}$ the pool of passages from which evidence is retrieved. The main experiments use the distractor setting of the three benchmarks, in which each question comes with its own pool containing both its supporting passages and distractors: between 17 and 20 passages (almost always 20) for MuSiQue, 10 for 2WikiMultihopQA, and at most 10 for HotpotQA. Section 4.5.2 additionally reports an open-corpus setting, in which $\mathcal{P}$ is a corpus shared by all questions of a dataset.

A decomposer maps Q to a sequence of sub-questions $(q_1, \ldots, q_K)$. A sub-question may contain placeholders of the form [Answer i], with i smaller than its own position, that refer to the answers of earlier hops, as in "Where was [Answer 1]

born?". Retrieval then proceeds hop by hop. At hop j, the placeholders in q_j are replaced by the answers $a_1, \ldots, a_{j-1}$ obtained at the earlier hops, which yields the expanded sub-question $\tilde{q}_j$. Every passage in the pool is scored by

$$s_j(p) = \cos\Big(E\big(\text{inst} \oplus \tilde{q}_j\big),\ E\big(\text{title}(p) \oplus \text{text}(p)\big)\Big), \qquad p \in \mathcal{P}, \tag{3.1}$$

where E is the BGE encoder, $\oplus$ denotes concatenation, inst is the instruction that BGE recommends prepending to retrieval queries, and a passage is encoded together with its title. The $k = 10$ highest-scoring passages form the candidate set $\mathcal{C}_j$; for 2WikiMultihopQA and HotpotQA, whose pools contain at most ten passages, $\mathcal{C}_j$ is the entire pool. One candidate $e_j \in \mathcal{C}_j$ is selected as the evidence of hop j. Without any further signal, the selection is simply

$$e_j = \arg\max_{p \in \mathcal{C}_j} s_j(p). \tag{3.2}$$

The LLM then generates a short answer a_j from the selected passage and the steps completed so far, and a_j is substituted into the later sub-questions that refer to it. After the last hop, the LLM produces the final answer from the expanded sub-questions, selected passages, and short answers of all hops.

This formulation makes the dependency between hops explicit. The evidence e_j determines the answer a_j, which enters every later sub-question that refers to it, so the candidates retrieved at later hops depend on the selections made earlier, and an error at one hop propagates to the rest of the chain. The method studied here keeps the decomposition, the retriever, and the answer generation of this pipeline and changes only the selection rule Equation 3.2, so that the selection also takes into account a risk score computed from the hidden states of the LLM. Table 3.1 summarizes the notation used in this chapter.

3.2 Gate-Guided Retrieval

This section describes how the evidence of each hop is selected with the risk score (Section 3.2.1), and how the chain is extended and the answers are generated (Section 3.2.2). Figure 3.1 gives an overview of one hop.

Symbol	Meaning	Defined in
Retrieval pipeline		
Q, $\mathcal{P}$	Multi-hop question; pool of passages	Section 3.1
K, q_j	Number of hops; sub-question of hop j, possibly with placeholders [Answer i]	Section 3.1
$\tilde{q}_j$, a_j	Sub-question with placeholders filled; short answer of hop j	Section 3.1
E, $s_j(p)$	BGE encoder; retrieval score of passage p at hop j	Equation 3.1
k, $\mathcal{C}_j$	Number of candidates ($k = 10$); candidate set of hop j	Section 3.1
e_j	Passage selected as the evidence of hop j	Equation 3.2
Gate-guided selection		
$x_{<j}$	Prefix: the question and the sub-questions and selected passages of earlier hops	Equation 3.3
g_j, $r_j(p)$	Gate of hop position j; risk score of candidate p	Equation 3.4
λ	Weight of the risk score ($\lambda = 0.6$)	Equation 3.5
Gate training and features		
j^*, y_j	Substituted hop of a counterfactual chain; label of hop j	Equation 3.6
e_j^*, p'	Annotated supporting passage of hop j; substituted passage	Section 3.3.1
ℓ	Layer of the LLM from which hidden states are read ($\ell = 15$)	Section 3.3.3
$\mathbf{u}_j$, $\mathbf{h}_j(p)$	Hidden state at the end of the prefix; at the end of candidate p	Equation 3.7
$\mathbf{\Delta}_j(p)$	Transition $\mathbf{h}_j(p) - \mathbf{u}_j$	Equation 3.8
Decomposer		
D, θ	Serialized decomposition; parameters of the decomposer	Equation 3.10

Table 3.1: Notation used in this chapter.

3.2.1 Candidate Scoring and Reranking

At hop j, the candidates $\mathcal{C}_j$ are retrieved as in Section 3.1. The risk score of a candidate is computed given the prefix of the chain, that is, the part of the chain that precedes hop j,

$$x_{<j} = \Big(Q, (q_1, e_1), \ldots, (q_{j-1}, e_{j-1})\Big), \tag{3.3}$$

which contains the question and, for every earlier hop, its sub-question and the text of its selected passage. The sub-questions are kept with their placeholders, and the

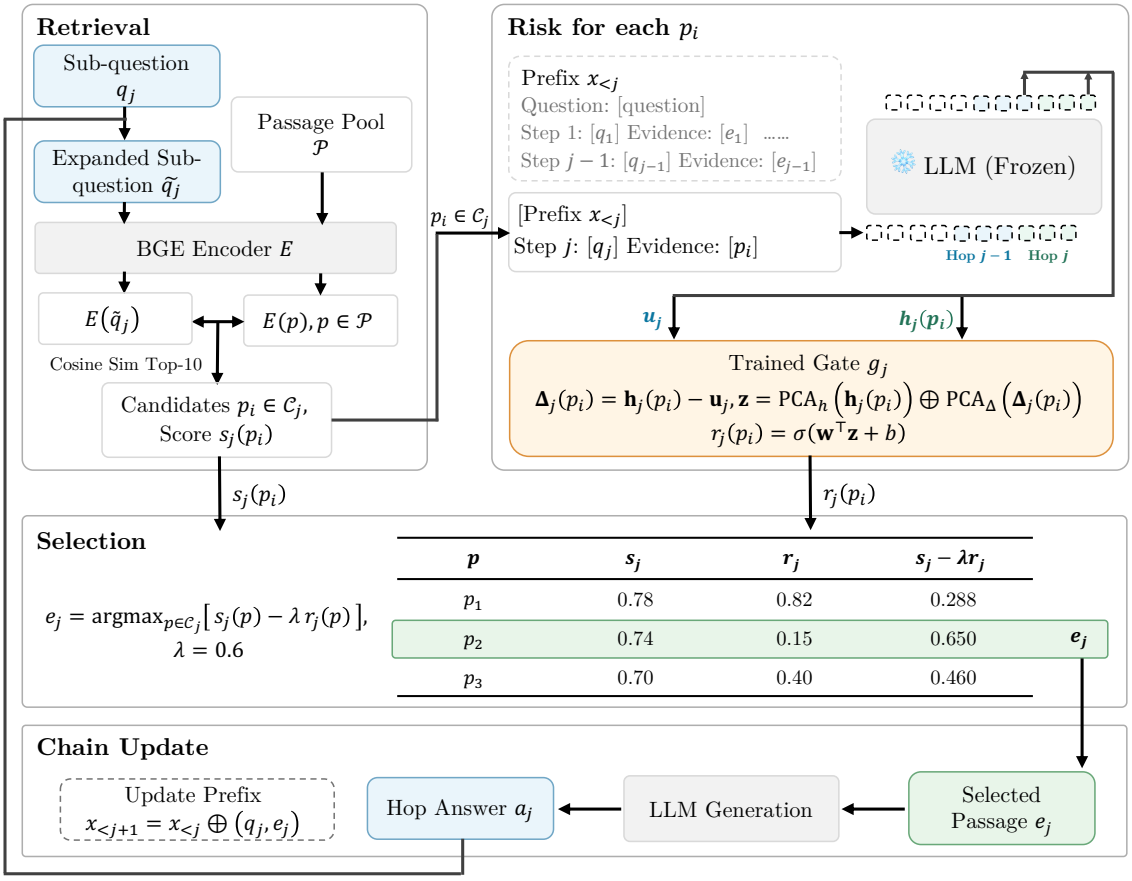


Figure 3.1: One hop of gate-guided retrieval. (1) Retrieval: the placeholders of the sub-question q_j are filled with the earlier answers, and the expanded sub-question $\tilde{q}_j$ is matched against the pool by the BGE encoder, giving the candidates $\mathcal{C}_j$ and their retrieval scores $s_j(p)$; for brevity, the retrieval instruction and the passage titles of Equation 3.1 are omitted. (2) Risk: for each candidate, the step "Step j: q_j Evidence: p" is appended to the prefix $x_{<j}$, with the placeholders of q_j left unfilled, and encoded by the frozen LLM; the gate g_j maps the state at the end of the candidate, $\mathbf{h}_j(p)$, and its difference from the state at the end of the prefix, $\mathbf{u}_j$, to a risk score $r_j(p)$. (3) Selection: the candidate with the highest $s_j(p) - \lambda r_j(p)$ is selected as e_j (values are illustrative). (4) Chain update: the LLM generates the answer a_j, which fills the placeholders of later sub-questions, and (q_j, e_j) is appended to the prefix for hop $j+1$.

short answers are not included. A gate assigns each candidate $p \in \mathcal{C}_j$ a risk score

$$r_j(p) = g_j\big(x_{<j},\, q_j,\, p\big) \in [0, 1], \tag{3.4}$$

computed from the hidden states of the LLM when the sub-question and the candidate are appended to the prefix; how the gate is trained, and hence what the

score measures exactly, is described in Section 3.3. Each candidate's retrieval score is adjusted by its risk score, and the candidate with the highest adjusted score is selected:

$$e_j = \arg\max_{p \in \mathcal{C}_j} \Big[s_j(p) - \lambda\, r_j(p) \Big], \tag{3.5}$$

which replaces Equation 3.2. Here $s_j(p)$ measures how relevant the candidate is to the expanded sub-question, $r_j(p)$ measures how strongly the hidden states of the chain resemble those of a substituted hop, and $\lambda \geq 0$ sets the trade-off between the two. With $\lambda = 0$ the selection reduces to retrieval alone, and larger values give the gate more influence; we use $\lambda = 0.6$, chosen on MuSiQue training questions.

The combination is a soft reranking rather than a hard filter: a candidate with a high risk score is pushed down the ranking but can still be selected if its retrieval score is much higher than those of the other candidates, and no candidate is discarded when all of them receive high risk scores. The cosine scores of Equation 3.1 and the risk scores both lie on bounded scales and are combined directly; for a retriever whose scores are unbounded, the scores of the candidates of each hop are first rescaled to $[0, 1]$ (Section 4.5.1).

3.2.2 Chain Update and Answer Generation

Once e_j is selected, the chain is updated before the next hop is processed. The LLM generates the answer a_j, which fills the placeholders of the later sub-questions that refer to hop j, and the pair (q_j, e_j) is appended to the prefix, which becomes $x_{<j+1}$. Answer generation is part of the pipeline of Section 3.1 rather than a separate component of the method: the same LLM and the same prompts are used whether the evidence was selected by Equation 3.2 or by Equation 3.5. Answers are generated with greedy decoding in two stages; the full prompts are given in Appendix B.

Answer of each hop. The LLM receives the question, the earlier steps (each with its expanded sub-question, selected passage, and answer), the expanded sub-question of hop j, and the title and text of the selected passage. It is asked to answer the sub-question with a short span, and to output "NA" if the passage does not contain the answer. Only the first line of the output is kept, and a long output

Algorithm 1: Gate-guided multi-hop retrieval for one question

Input: question Q; passage pool $\mathcal{P}$; sub-questions $(q_1, \ldots, q_K)$ from the decomposer; gates $g_1, \ldots, g_4$; weight λ; number of candidates k

Output: evidence chain $(e_1, \ldots, e_K)$ and final answer

$x_{<1} \leftarrow (Q)$;
for $j \leftarrow 1$ **to** K **do**
  $\tilde{q}_j \leftarrow q_j$ with each placeholder [Answer i] replaced by a_i;
  $\mathcal{C}_j \leftarrow$ the k passages of $\mathcal{P}$ with the highest $s_j(p)$; // Equation 3.1
  foreach $p \in \mathcal{C}_j$ **do**
    if $j \leq 4$ **then**
      $r_j(p) \leftarrow g_j(x_{<j}, q_j, p)$; // Equation 3.4
    else
      $r_j(p) \leftarrow 0$;
  $e_j \leftarrow \arg\max_{p \in \mathcal{C}_j} [s_j(p) - \lambda\, r_j(p)]$; // Equation 3.5
  $a_j \leftarrow$ short answer generated by the LLM for $\tilde{q}_j$ from e_j and the earlier steps;
  $x_{<j+1} \leftarrow x_{<j} \oplus (q_j, e_j)$;
final answer $\leftarrow$ generated by the LLM from $(\tilde{q}_j, e_j, a_j)_{j=1}^{K}$ and Q;
return $(e_1, \ldots, e_K)$, *final answer*

that states that the information is missing is mapped to "NA".

Final answer. After the last hop, the LLM receives the full chain, consisting of the expanded sub-question, selected passage, and answer of every hop, followed by the original question. A single prompt is used for all questions. It asks for a short answer and includes one conditional instruction for comparison questions: if the question asks which of several entities satisfies a comparison, the answers of the individual hops are only the values being compared, and the answer must name the entity. The LLM is asked to output a single line of the form "Final answer: <answer>"; the last such line is taken as the final answer, and if none is found, the answer of the last hop is used instead.

Algorithm 1 summarizes the complete procedure for one question.

3.3 Hidden-State Gate

The method relies on two trained components: the gate, which provides the risk score $r_j(p)$ used in Equation 3.5, and the decomposer, which provides the sub-questions. They are trained separately, on different data and with different supervi-

Component	Training data	Supervision	Used at inference
Decomposer	Annotated MuSiQue decompositions and generated 2WikiMultihopQA decompositions	Question → sequence of sub-questions	All three benchmarks
Gate	MuSiQue annotated evidence and counterfactual chains	Hop-level replacement label	All three benchmarks

Table 3.2: The two trained components of the method.

sion, and both are applied to all three benchmarks at inference time, as summarized in Table 3.2. The gate is trained only on MuSiQue, because only MuSiQue provides hop-level supporting-paragraph annotations (Section 2.1). Training chains are always built from each question's own passage pool, whatever retrieval setting is used at inference time. This section describes how chains with a known substituted hop are constructed (Section 3.3.1), how the training data are augmented and balanced (Section 3.3.2), and how the gate turns the hidden states of the LLM into a risk score (Section 3.3.3).

3.3.1 Counterfactual Trace Construction

Each MuSiQue question is annotated with one single-hop question per hop, its answer, and the index of its supporting paragraph. Because the sub-questions produced by the decomposer are natural-language questions with placeholders [Answer i], we rewrite the annotated hops into the same form with GPT-4.1-mini, followed by a verification pass. The supporting paragraph and the answer of each hop are kept unchanged.

For every training question, we construct one gold chain and a set of counterfactual chains. A gold chain pairs every sub-question with the paragraph annotated as its support. A counterfactual chain is identical except at a single hop j^*, where the annotated paragraph is replaced by another passage from the question's own pool. The replacement passages are chosen to be hard negatives. For hop j^*, we fill the placeholders of the sub-question with the annotated answers of the ear-

lier hops, score all passages in the pool with the BGE encoder, and take the ten highest-scoring passages. This scoring follows Equation 3.1 except that the query is encoded without the retrieval instruction. Passages annotated as supporting any hop of the question are skipped, and the three highest-ranked remaining passages are each used to form one counterfactual chain; every pool contains at least three such passages, so every hop of every question yields exactly three counterfactual chains. The substituted passages are therefore those that the retriever itself ranks highly for the sub-question, which resembles the errors made at inference time far more closely than randomly sampled passages would. We construct errors in this controlled way, rather than collecting the errors that the system makes naturally, because the position of a natural error is not known reliably, whereas a constructed chain specifies exactly which hop was altered.

Each gold chain contributes one training example at every hop. A counterfactual chain with substituted hop j^* contributes examples only for hops $1, \ldots, j^*$; its later hops are discarded. An example has the same form as the input of the gate at inference time: the prefix $x_{<j}$ of Equation 3.3, built from the annotated passages of the earlier hops, together with the sub-question q_j and the passage of hop j. The final answer of the question is never part of an example. The label of hop j is

$$y_j = \begin{cases} 1 & \text{if the chain is counterfactual and } j = j^*, \\ 0 & \text{otherwise.} \end{cases} \tag{3.6}$$

These labels are determined by construction: $y_j = 1$ records that the evidence at hop j was substituted for the annotated supporting passage, rather than a separate judgment of whether that passage is factually wrong. Figure 3.2 illustrates the chains and the examples they yield. Because a counterfactual chain is cut off at j^*, every training prefix $x_{<j}$ consists of annotated evidence only, and a substituted passage appears in an example exactly when $y_j = 1$. The gate receives no explicit indication of whether the prefix is correct; it is trained only to predict y_j. At inference time, the prefix consists of the passages selected at the earlier hops and may contain errors, a situation that does not occur in the training data. How the score behaves in that situation, and how this behavior relates to the way the training data are constructed, is examined in Section 5.1.

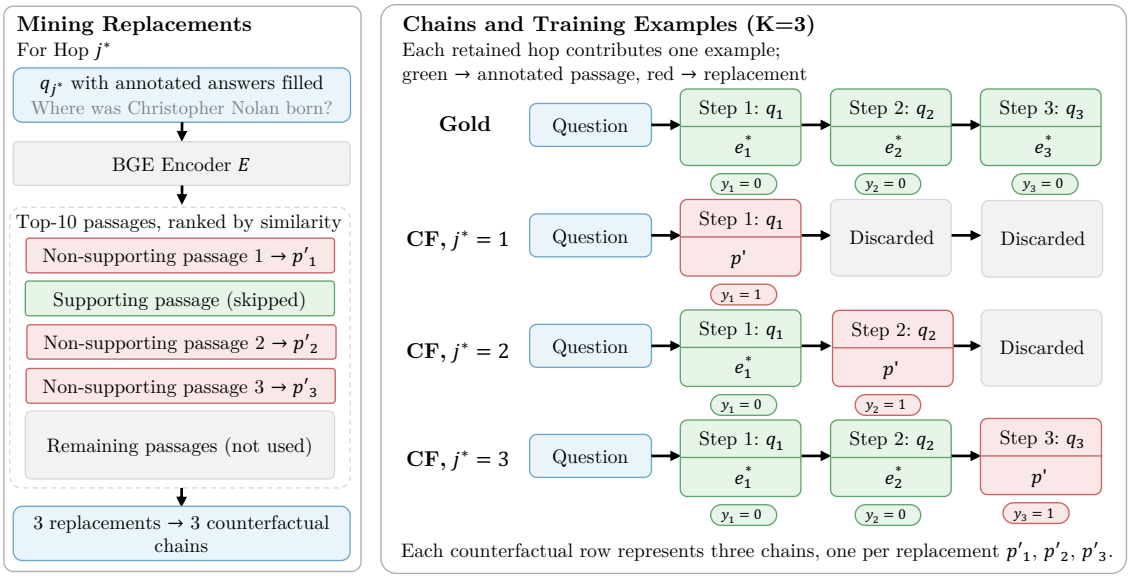


Figure 3.2: Counterfactual trace construction for a three-hop question. Left: for the substituted hop j^*, the sub-question is filled with the annotated answers of the earlier hops, the pool is ranked by the retriever, passages annotated as supporting any hop are skipped, and the three highest-ranked remaining passages of the top ten are used as replacements. Right: the gold chain (top) and the counterfactual chains for each $j^* \in \{1, 2, 3\}$; each counterfactual row stands for three chains, one per replacement. Annotated passages (e_j^*) are shown in green, substituted passages (p') in red, and hops after j^*, which are discarded, in gray. Each remaining hop yields one example, consisting of the prefix and the step of that hop, with the label y_j shown below it. In every example the prefix contains only annotated passages, and a substituted passage appears exactly in the examples with $y_j = 1$.

3.3.2 Gate Training Data and Balancing

Long chains are scarce in MuSiQue: among the 19,938 training questions, 14,376 have two hops, 4,387 three, and only 1,175 four. To provide more training examples for the later hop positions, we create paraphrased variants of the three- and four-hop questions with the LLM. These variants are used only to build training chains for the gate; the decomposer does not use them. Each hop of the natural-language decomposition is paraphrased separately with its placeholders preserved. A variant is accepted only if it passes three checks: it has the same number of hops as the original and the same pattern of [Answer i] references at every hop; each hop refers only to the answers of earlier hops; and no annotated answer of any hop, of at least

		Train	Dev
Chains	Gold	24,070	2,417
	Counterfactual (before balancing)	179,571	19,212
	Counterfactual (after balancing)	59,857	6,404
	Total used	83,927	8,821
Examples per hop position	Hop 1	83,927	8,821
	Hop 2	59,857	6,404
	Hop 3	21,411	2,735
	Hop 4	4,046	810

Table 3.3: Gate-training traces and resulting hop-level examples. The training split includes the paraphrased variants of three- and four-hop questions; only the balanced chains are used. Each chain contributes one example to every hop position it reaches, so the number of examples decreases with the hop position.

three characters, appears in any of its sub-questions, which prevents a paraphrase from revealing an answer that the chain is meant to retrieve. Of 5,562 candidate variants, 4,132 pass these checks. Each accepted variant is treated as an additional training question with the same supporting paragraphs and answers as the original, which raises the number of three- and four-hop training questions to 7,671 and 2,023.

Because a question with K hops yields one gold chain but $3K$ counterfactual chains, counterfactual chains are far more numerous than gold chains. We therefore balance them by question length K and substituted hop j^*: for each pair (K, j^*), we sample without replacement as many counterfactual chains as there are gold chains with K hops. This procedure matches the number of gold and counterfactual chains within each (K, j^*) stratum, but it does not change how scarce the later hop positions are. Since every chain contributes one example to each hop position it reaches, the training data contain 83,927 examples at the first hop position but only 4,046 at the fourth. Table 3.3 summarizes the resulting data. The development split of MuSiQue is processed in the same way, without paraphrased variants; these development chains are used only to evaluate the gate as a classifier, whereas the end-to-end evaluation in Chapter 4 uses the same development questions but none of these chains.

Text	Content
Prefix of hop 1	Question: Where was the director of Inception born?
Hop 1, candidate p	Question: Where was the director of Inception born? **Step 1: Who directed Inception? Evidence:** [p]
Hop 2, candidate p	Question: Where was the director of Inception born? Step 1: Who directed Inception? Evidence: [e_1] **Step 2: Where was [Answer 1] born? Evidence:** [p]

Table 3.4: Texts read by the gate for the example of Figure 1.1. Each hop appends one step (in bold) to the prefix. The second sub-question keeps its placeholder, and the answer of the first hop does not appear.

3.3.3 Hidden-State Features and Gate Scoring

Serialization. Following the step-marked format of Sun et al. [13], a chain with K hops is written as text in the following form (shown with one step per line for readability):

Question: [question] [pos 0]
Step 1: [sub-question 1] Evidence: [passage 1] [pos 1]
Step 2: [sub-question 2] Evidence: [passage 2] [pos 2]
...
Step K: [sub-question K] Evidence: [passage K] [pos K]

Each step contains the sub-question with its placeholders and the text of the passage without its title. Table 3.4 shows the texts read for the example of Figure 1.1.

Hidden states. To score a candidate p at hop j, the step "Step j: [sub-question j] Evidence: [p]" is appended to the prefix $x_{<j}$. The gate reads two hidden states of the LLM at layer ℓ:

$$\mathbf{u}_j = \text{state at [pos } j-1\text{]}, \qquad \mathbf{h}_j(p) = \text{state at [pos } j\text{] with } p \text{ appended}, \tag{3.7}$$

that is, the state at the end of the prefix, which is the same for all candidates at hop j, and the state at the end of the candidate. Each state is read from a separate forward pass: the text up to the marker is rendered as a single complete user turn

of the LLM's chat template, without a generation prompt, and the hidden state of its final token, the end-of-turn token, is taken.

We use layer $\ell = 15$, counting from zero, of the 32 layers of Llama-3.1-8B-Instruct, that is, the output of its sixteenth transformer layer. The layer was selected by comparing layers 7, 15, 23, and 31 on a held-out part of the training data: the training questions were divided into two groups, the gate described below was fitted on one, and its area under the ROC curve was measured on the other, which contains 20% of the questions. Layer 15 performed best at all four hop positions; the comparison is reported in Section 4.4.2. The development data were not used.

Features. From the two hidden states in Equation 3.7, the gate forms two features for each candidate:

$$\mathbf{h}_j(p) \qquad \text{and} \qquad \mathbf{\Delta}_j(p) = \mathbf{h}_j(p) - \mathbf{u}_j. \tag{3.8}$$

The accumulated state $\mathbf{h}_j(p)$ summarizes the whole chain once the candidate has been added, whereas the transition $\mathbf{\Delta}_j(p)$ captures the change that the sub-question and candidate of hop j induce relative to the prefix. Combining the state at a step with its change from the previous step follows the trajectory features of Sun et al. [13].

Gate scoring. The gate uses a separate model g_j for each hop position $j \in \{1, 2, 3, 4\}$, fitted on all training examples at that position, whatever the number of hops of the question they come from. Each g_j is a linear classifier on reduced features. The accumulated state and the transition are reduced separately with principal component analysis, each to 64 dimensions, because the two features have different covariance structures, and the two reduced vectors are concatenated. A logistic regression then gives

$$r_j(p) = \sigma\Big(\mathbf{w}_h^\top \operatorname{PCA}_h\big(\mathbf{h}_j(p)\big) + \mathbf{w}_\Delta^\top \operatorname{PCA}_\Delta\big(\mathbf{\Delta}_j(p)\big) + b\Big), \tag{3.9}$$

where σ is the logistic function. Both projections and the logistic regression are fitted on the examples of position j only, with an inverse regularization strength of 1 and class weights that compensate for the imbalance between the two labels. We use a linear model deliberately: a probe that is too expressive may learn the task

itself rather than reveal information present in the hidden states [26]. Fitting all four models takes a few minutes on a CPU.

The annotated decompositions contain at most four hops, so no model exists for a fifth position. The decomposer occasionally predicts five hops (14 of the 22,398 development questions across the three benchmarks); the fifth hop of such a question receives no risk score and is ranked by Equation 3.2. For offline evaluation and for the analysis in Chapter 5, each g_j is also given a decision threshold, set on the training data so that 15% of the examples with label 0 exceed it. The threshold plays no role in retrieval, which uses the score $r_j(p)$ itself.

Inference cost. The gate adds no generation and no model other than the LLM already used for the answers. Its cost consists of forward passes of the LLM without decoding, each on a text that contains the prefix and at most one candidate. The prefix of hop j is identical to the text read for the passage selected at hop $j-1$, so its state $\mathbf{u}_j$ is already available from the previous hop, and only the prefix of the first hop needs a separate pass. For a question with K hops, the gate therefore requires $1 + \sum_j |\mathcal{C}_j|$ forward passes, at most $1 + 10K$. The passes of a hop are independent of one another and are batched, and they run separately from answer generation. The decomposition is computed once per question and is shared with the pipeline without the gate.

3.4 Question Decomposer

At inference time, no annotated decomposition is available for any of the three benchmarks, so the sub-questions of Section 3.1 are produced by a trained decomposer. Training it requires pairs of a question and its decomposition. MuSiQue provides such pairs, 2WikiMultihopQA does not, and HotpotQA is not used for training. Unlike the gate, the decomposer is therefore trained on MuSiQue together with decompositions that we generate for 2WikiMultihopQA. This section describes this supervision (Section 3.4.1) and how the decomposer is trained and applied (Section 3.4.2).

3.4.1 Decomposition Supervision and Target Format

MuSiQue annotates every question with one single-hop question per hop. We use these decompositions in a serialized form as the training target: the hops are written in order, each hop is marked with its position, and a reference to the answer of an earlier hop is written as the index of that hop. A hop is either a natural-language question or a compact entity–relation pair, such as the entity "The Secret of the Swamp" with the relation "director". We use all 19,938 training questions of MuSiQue.

2WikiMultihopQA annotates its questions with evidence triples but provides no decompositions, and MuSiQue contains no comparison questions, which ask which of several entities satisfies a condition. To cover this question type, we sample 10,444 training questions of 2WikiMultihopQA by question type, with an emphasis on comparison and bridge-comparison questions, and generate a decomposition for each with GPT-5-mini in the same serialized form. The resulting 30,382 pairs train the decomposer only. They use the serialized form rather than the natural-language rewrite of Section 3.3.1, and they include none of the paraphrased variants of Section 3.3.2.

3.4.2 Decomposer Training and Inference

The decomposer is a BART-large model [37]. Its input is the question Q and its output is the decomposition $D = (q_1, \ldots, q_K)$, serialized as a token sequence $d_1, \ldots, d_T$. It is trained by minimizing the negative log-likelihood of the target sequence,

$$\mathcal{L}_{\text{dec}}(\theta) = -\sum_{(Q,D)} \sum_{t=1}^{T} \log p_\theta\Big(d_t \mid d_{<t}, Q\Big), \tag{3.10}$$

where θ denotes the parameters of the model and the outer sum runs over the training pairs; label smoothing of 0.1 is applied. The model is trained for three epochs with a learning rate of 3×10^{-5} and a batch size of 16, and the checkpoint with the highest BLEU score on development decompositions of MuSiQue and 2WikiMultihopQA is kept.

At inference time, the decomposition of each question is generated with beam search (beam size 10). Because a hop in the serialized form may be an entity–relation pair rather than a question, the LLM rewrites every hop into a natural-

language sub-question, followed by a verification pass, and references to earlier answers are written as placeholders [Answer i]. The same decomposer is used for all three benchmarks, including HotpotQA, from which it saw no training data.

Chapter 4

Experiments and Results

This chapter evaluates the method of Chapter 3. Section 4.1 describes the datasets, the implementation, and the evaluation metrics. Section 4.2 examines the premises of the method and its two trained components in isolation: the structure of the hidden states along a retrieval chain, the gate as a classifier, and the decomposer. Section 4.3 reports the main end-to-end results against a control that differs from the method only in the selection rule. Section 4.4 studies the design choices of the gate and the decomposer, Section 4.5 tests the method with a different retriever and in an open corpus, and Section 4.6 compares it with two recent multi-hop retrieval systems.

4.1 Experimental Setup

This section describes the data on which the method is evaluated (Section 4.1.1), how it is implemented and configured (Section 4.1.2), and how its outputs are scored (Section 4.1.3).

4.1.1 Datasets

We evaluate on the development sets of MuSiQue [4], 2WikiMultihopQA [3], and HotpotQA [2]. Table 4.1 summarizes them. The main experiments use the distractor setting, in which each question comes with its own pool of passages; Section 4.5.2 additionally uses an open corpus.

In tables, 2WikiMultihopQA is abbreviated as 2Wiki. The number of annotated hops differs in kind across the three datasets. MuSiQue annotates a decomposition with two to four hops. For 2WikiMultihopQA and HotpotQA, we take the number of distinct titles among the supporting facts as the number of hops. In 2Wiki-

Dataset	Questions	Passages per question	Annotated hops (questions)
MuSiQue	2,417	17–20	2 (1,252), 3 (760), 4 (405)
2Wiki	12,576	10	2 (9,825), 4 (2,751)
HotpotQA	7,405	$\leq$10	2 (7,405)

Table 4.1: Development sets used for evaluation. In 2WikiMultihopQA, the four-hop questions are exactly the bridge comparison questions (2,751); the remaining questions are compositional (5,236), comparison (3,040), and inference (1,549) questions. HotpotQA contains 5,918 bridge and 1,487 comparison questions.

MultihopQA this number is determined by the question type: bridge comparison questions have four hops and all other types have two. Every HotpotQA question has two.

The three datasets play different roles. The gate is trained only on MuSiQue, and the decomposer only on MuSiQue and on decompositions that we generated for 2WikiMultihopQA (Section 3.4). No component of the method is trained on HotpotQA data, so HotpotQA measures how well the method transfers to a dataset it has never seen.

4.1.2 Implementation Details

The retriever is BAAI/bge-base-en-v1.5 [35], the LLM is Llama-3.1-8B-Instruct [36] in bfloat16, and the decomposer is BART-large [37]. Answers are generated with greedy decoding, with at most 64 new tokens for the answer of a hop and 128 for the final answer. The hidden states read by the gate are computed by a separate copy of the LLM that only runs forward passes without decoding.

Throughout this chapter, the method is compared with a *control* that uses the same decomposer, retriever, prompts, and code, and differs only in the selection rule: it sets $\lambda = 0$ in Equation 3.5, so that each hop selects the candidate with the highest retrieval score as in Equation 3.2. The difference between the method and the control is therefore the effect of the gate alone.

Table 4.2 lists the hyperparameters of the method and how each was set. None of them was chosen on the development questions used for the end-to-end evaluation, with one indirect exception: the decomposer checkpoint was selected by the BLEU score of its decompositions on a set that includes MuSiQue development questions,

Hyperparameter	Value	How it was set
Hidden layer ℓ	15	Compared with layers 7, 23, and 31 on held-out training questions
PCA dimensions	64 + 64	Fixed
Logistic regression	$C = 1$, balanced classes	Fixed
Candidates per hop k	10	Fixed
Weight λ	0.6	Swept on 3,000 MuSiQue training questions; results flat between 0.4 and 0.7
Decision threshold	15% of label-0 examples above it	Set on training data; used only for offline evaluation and analysis
Decomposer checkpoint	best BLEU	Development decompositions of MuSiQue and 2Wiki

Table 4.2: Hyperparameters of the method and how they were set.

which concerns only the wording of the decompositions, not retrieval or answer accuracy.

4.1.3 Evaluation Metrics

We evaluate retrieval at the level of hops and of whole chains, and the final answers.

Hop-level retrieval. For a hop j, recall@k indicates whether the passage annotated for hop j is among the k highest-ranked candidates after reranking, computed over the full candidate set $\mathcal{C}_j$; we report recall@1 and recall@3, averaged over hops. Hops are matched by position, so a hop is scored only if both the predicted and the annotated decomposition contain it. When the decomposer predicts fewer hops than annotated, the missing hops are therefore not counted. This affects 5.7% of the annotated hops of MuSiQue, mostly from four-hop questions decomposed into fewer hops, and about 0.1% in the other two datasets. Where the number of predicted hops differs between the compared systems (Section 4.4.5), we instead count every annotated hop and treat missing hops as misses.

Chain-level retrieval. Chain@1 indicates whether the selected passages are correct at every hop of a question. We report two versions. The lenient version compares the predicted and annotated hops position by position as far as both extend; it measures evidence selection given the decomposition. The strict version additionally requires the number of predicted hops to equal the number of annotated hops, and so also counts the errors of the decomposer in the number of hops.

Answers. Exact match (EM) and token-level F1 follow the normalization of the official HotpotQA evaluation. For MuSiQue, the prediction is compared with the annotated answer and its aliases, and the best score is taken, as in the official MuSiQue evaluation; 2WikiMultihopQA and HotpotQA provide a single answer.

Significance. Differences in EM are tested per question with an exact binomial McNemar test, and 95% confidence intervals are obtained from 2,000 paired bootstrap resamples.

4.2 Component Evaluation

Before turning to end-to-end results, this section examines the premises of the method and its two trained components separately. Section 4.2.1 asks whether the hidden states read along a retrieval chain are organized by position and whether they shift when a hop receives a substituted passage. Section 4.2.2 evaluates the gate as a classifier, and Section 4.2.3 evaluates the decomposer.

4.2.1 Hidden-State Structure

The analyses in this subsection use the states at layer 15 of the gold and counterfactual chains of Section 3.3.1, read at the end of every prefix as in Section 3.3.3. A chain with K hops has $K + 1$ prefixes, from the question alone (prefix index 1) to the full chain (prefix index $K + 1$).

Position. Figure 4.1 projects the states of 1,000 gold training chains for each chain length with t-SNE [38]. Within each panel, the states of each prefix index form a separate cluster; in the four-hop panel, prefix indices 4 and 5 lie closest to each

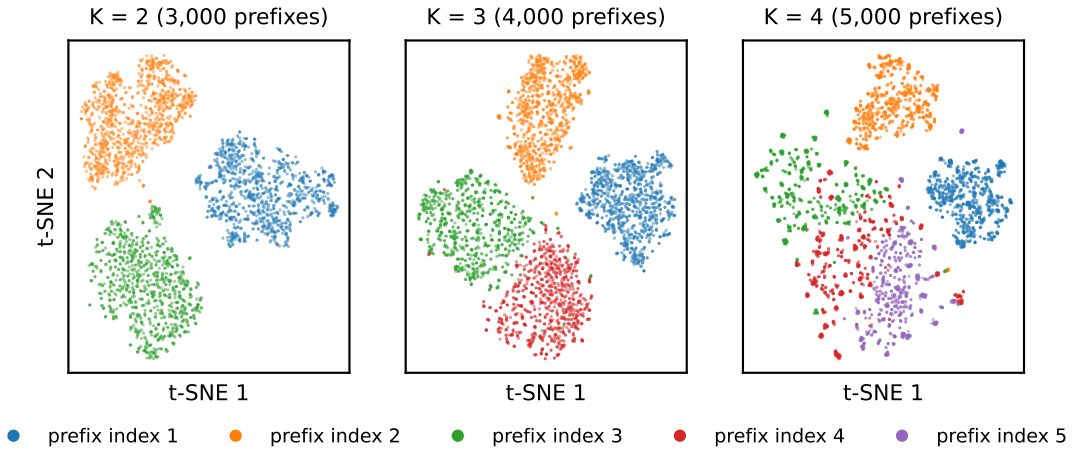


Figure 4.1: t-SNE projection of the layer-15 states of gold chains, one panel per chain length K, each with 1,000 training chains and all of their prefixes. Colors indicate the prefix index, from the question alone (1) to the full chain ($K+1$). The axes of a t-SNE projection have no units.

Chain length	Test prefixes	Hidden state	Token count only	Majority class
$K = 2$	3,756	1.000	0.870	0.333
$K = 3$	3,040	0.999	0.827	0.250
$K = 4$	2,025	0.985	0.677	0.200

Table 4.3: Accuracy of predicting the prefix index of gold development chains, with a linear classifier trained on training chains. "Token count only" uses the length of the prefix as its single feature.

other but remain distinguishable. Because a t-SNE projection is only illustrative, we quantify the structure in two ways. First, among the ten nearest neighbors of a state, taken from chains of other questions, 99.95%, 97.9%, and 97.5% have the same prefix index for $K = 2$, 3, and 4. Second, a multinomial logistic regression trained on the training chains predicts the prefix index of the development chains with the accuracies in Table 4.3.

Because later prefixes are longer, the position could in principle be recovered from length alone. A classifier that sees only the number of tokens reaches 0.68 to 0.87, well below the hidden states. Within narrow length ranges, obtained by splitting the prefixes into quintiles of their token count, the classifier on hidden states still reaches 0.96 to 1.00, above the share of the most frequent prefix index in

every range. These results show that the states at layer 15 encode the position of the prefix linearly, in line with the step-specific regions reported by Sun et al. [13] for mathematical reasoning. They do not show that each position has its own reasoning geometry: the serialized text contains the step markers themselves, from which the position can be read off. The observation supports, nonetheless, the use of a separate gate for each hop position (Section 3.3.3).

Substituted evidence. The second premise is that a substituted passage moves the state in a detectable way. For each chain length K and substituted hop j^*, we take 400 development questions and compare, at the prefix that first contains hop j^*, the state of the gold chain with that of a counterfactual chain of the same question. We estimate a single direction as the difference between the mean states of the two groups on one half of the pairs, and project the states of the other half onto it. Figure 4.2 shows the resulting scores. In all nine combinations of K and j^*, the projection separates the two groups with an area under the ROC curve (AUC) between 0.89 and 0.93 on the held-out half. A single linear direction, estimated without any classifier, thus separates gold from substituted evidence, which is the signal that the gate builds on. These AUCs are of the same order as those of the gate (Section 4.2.2), but the two are not directly comparable, since here each combination is evaluated separately on balanced pairs.

4.2.2 Gate

We evaluate the gate as a binary classifier on the development chains of MuSiQue (8,821 chains, Table 3.3), with the decision threshold that was set on the training data. Table 4.4 reports the results. Over all hop positions, the gate reaches an AUC of 0.936, with a true positive rate of 0.884 at a false positive rate of 0.182. The false positive rate exceeds the target of 0.15 because the threshold is transferred from the training data without adjustment; the deviation is largest at the fourth position (0.274), which has the fewest examples both in training (4,046) and in development (810). The first position is detected most reliably, and the third least reliably.

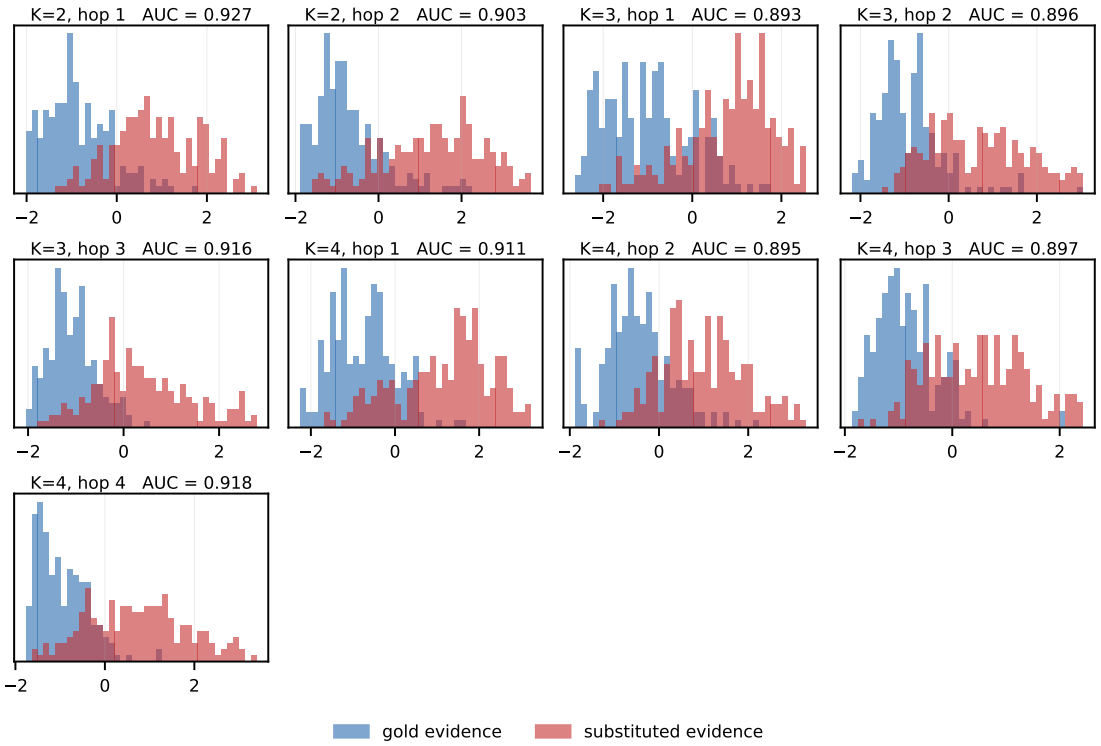


Figure 4.2: Projection of development states onto the mean difference between substituted and gold evidence, for each chain length K and substituted hop. The direction is estimated on one half of 400 question pairs, and the histograms and the AUC in each title are computed on the other half. Blue: gold evidence; red: substituted evidence.

Hop position	Development examples	True positive rate	False positive rate
1	8,821	0.934	0.173
2	6,404	0.873	0.178
3	2,735	0.803	0.208
4	810	0.884	0.274
All	18,770	0.884	0.182

Table 4.4: The gate as a classifier on the MuSiQue development chains, at the threshold set on the training data. The AUC over all positions is 0.936 (precision 0.715, F1 0.791).

4.2.3 Decomposer

The decomposer reaches a BLEU score [39] of 76.81 on the MuSiQue development decompositions. Because the sub-questions are rewritten into natural lan-

Dataset	Group	Questions	Correct number of hops
MuSiQue	2 hops	1,252	0.982
	3 hops	760	0.782
	4 hops	405	0.489
	All	2,417	0.837
2Wiki	Compositional (2 hops)	5,236	0.999
	Comparison (2 hops)	3,040	0.957
	Inference (2 hops)	1,549	1.000
	Bridge comparison (4 hops)	2,751	0.985
	All	12,576	0.986
HotpotQA	All (2 hops)	7,405	0.876

Table 4.5: Proportion of development questions for which the decomposer predicts the annotated number of hops.

guage before use, the number of hops matters more than their wording. Table 4.5 reports how often the predicted number of hops equals the annotated one. On MuSiQue, two-hop questions are almost always decomposed correctly (0.982), but only 0.489 of the four-hop questions are; most of the others are decomposed into three hops. On 2WikiMultihopQA, the number of hops is correct for 98.6% of the questions, including the four-hop bridge comparison questions (0.985); the lowest rate is on two-hop comparison questions (0.957). On HotpotQA, on which the decomposer was not trained, 87.6% of the questions receive two hops, and 902 questions (12.2%) receive three or more. These errors in the number of hops are what separates the lenient and the strict versions of chain@1 in the following sections.

4.3 Main Results

Table 4.6 compares the method with the control on the three datasets. The gate improves every metric on every dataset. Recall@1 rises by 9.1, 9.7, and 8.8 points on MuSiQue, 2WikiMultihopQA, and HotpotQA, and the improvement at the level of whole chains is larger: lenient chain@1 rises by 15.2, 19.3, and 17.1 points, and strict chain@1 by 15.3, 19.3, and 15.8 points. Because both systems use the same decomposer, the choice between the two versions of chain@1 hardly affects the improvement, although the strict values are lower, by up to 6.4 points on HotpotQA. The larger gain at the chain level is consistent with the improvements

		Recall		Chain@1			
Dataset	Selection	@1	@3	Lenient	Strict	EM	F1
MuSiQue	Control	0.609	0.768	0.361	0.345	0.329	0.414
	Gate	**0.700**	**0.844**	**0.513**	**0.498**	**0.441**	**0.536**
2Wiki	Control	0.833	0.915	0.669	0.658	0.521	0.586
	Gate	**0.930**	**0.985**	**0.863**	**0.851**	**0.601**	**0.681**
HotpotQA	Control	0.575	0.774	0.390	0.339	0.460	0.583
	Gate	**0.662**	**0.870**	**0.561**	**0.497**	**0.552**	**0.688**

Table 4.6: Main results on the development sets. The control differs from the method only in the selection rule ($\lambda = 0$, selection by retrieval score). The gate is trained on MuSiQue only.

Dataset	ΔEM	95% CI	Gate only	Control only	McNemar p
MuSiQue	+11.21	[+9.64, +12.74]	355	84	1.1×10^{-40}
2Wiki	+8.02	[+7.47, +8.60]	1,170	162	1.1×10^{-188}
HotpotQA	+9.11	[+8.33, +9.94]	847	172	1.2×10^{-107}

Table 4.7: Paired comparison of EM between the method and the control, in points. The middle columns count the questions answered correctly by only one of the two.

of individual hops accumulating along the chain, since a chain is correct only if all of its hops are. EM rises by 11.2, 8.0, and 9.1 points and F1 by 12.2, 9.5, and 10.5 points. The gains on 2WikiMultihopQA and HotpotQA are of the same size as on MuSiQue, although the gate was trained only on MuSiQue.

Table 4.7 tests the differences in EM question by question. All three are significant ($p < 10^{-40}$), and the 95% confidence intervals exclude values below 7.4 points. The gate also changes some answers from correct to wrong: the control answers correctly and the method incorrectly for 84, 162, and 172 questions, or 3.5%, 1.3%, and 2.3% of the development sets. They are outnumbered by the questions that change in the opposite direction by a factor of four to seven.

Both systems use the same final-answer prompt, including its instruction for comparison questions (Section 3.2.2), so the differences above are due to the gate alone. The contribution of that instruction is examined separately in Section 4.4.4.

4.4 Ablation Studies

This section examines the design choices of the method. The first three subsections concern the gate: its features (Section 4.4.1), the layer from which they are read (Section 4.4.2), and the weight λ (Section 4.4.3). The last two concern the rest of the pipeline: the final-answer prompt (Section 4.4.4) and the training data of the decomposer (Section 4.4.5).

4.4.1 Gate Features

The gate combines the accumulated state $\mathbf{h}_j(p)$ with the transition $\boldsymbol{\Delta}_j(p)$, following Sun et al. [13], whose most predictive features combine the state at the end of the reasoning with its change over the last step. To see how much each feature contributes in multi-hop retrieval, we compare the gate with gates trained on either feature alone, with the same layer, dimensions, classifier, and weight λ. Table 4.8 shows that the three variants are ordered in the same way on every dataset and every metric: the transition alone is weakest, the accumulated state alone is close to the combination, and the combination is best. The same order appears in the offline AUC on the MuSiQue development chains, 0.920, 0.935, and 0.936. Each feature carries most of the signal on its own; relative to the control, the transition alone already raises EM by 6.7 to 10.4 points. Once the accumulated state is available, however, adding the transition improves EM by only 0.2 to 0.3 points. The combination is thus best, as in the study of Sun et al., but in this setting nearly all of its advantage over the transition alone is already provided by the accumulated state. Which feature drives the decisions of the combined gate is analyzed in Chapter 5.

4.4.2 Layer Selection

The layer of the LLM from which the gate reads its features was chosen on the training data. The training questions of MuSiQue were split into a fitting set and a held-out set containing 20% of the questions, so that all chains of a question fall on the same side; the gate was fitted on the first and evaluated on the second, at layers 7, 15, 23, and 31. Table 4.9 shows that layer 15 gives the highest AUC at all four hop positions. It exceeds the last layer by 1.05 to 1.35 points, and the earliest candidate, layer 7, falls 7.5 to 12.8 points below it. Layer 23 is close to layer 15

Dataset	Features	Recall@1	Chain@1 (lenient)	Chain@1 (strict)	EM
MuSiQue	Control	0.609	0.361	0.345	0.329
	$\mathbf{\Delta}$ only	0.687	0.481	0.464	0.433
	$\mathbf{h}$ only	0.698	0.510	0.495	0.439
	Both	**0.700**	**0.513**	**0.498**	**0.441**
2Wiki	Control	0.833	0.669	0.658	0.521
	$\mathbf{\Delta}$ only	0.914	0.831	0.819	0.588
	$\mathbf{h}$ only	0.929	0.860	0.848	0.599
	Both	**0.930**	**0.863**	**0.851**	**0.601**
HotpotQA	Control	0.575	0.390	0.339	0.460
	$\mathbf{\Delta}$ only	0.659	0.541	0.477	0.547
	$\mathbf{h}$ only	0.661	0.551	0.488	0.549
	Both	**0.662**	**0.561**	**0.497**	**0.552**

Table 4.8: Gates trained on the transition only, the accumulated state only, and both, with all other settings fixed.

Hop position	Examples	Layer 7	Layer 15	Layer 23	Layer 31
1	83,927	0.894	**0.969**	0.965	0.958
2	59,857	0.844	**0.940**	0.936	0.926
3	21,411	0.791	**0.918**	0.915	0.907
4	4,046	0.840	**0.945**	0.928	0.931

Table 4.9: AUC of the gate on held-out training questions for each candidate layer (counting from zero). The development data were not used.

at the later positions but does not exceed it. The gate therefore reads its features from a middle layer rather than from the last one.

4.4.3 Fusion Weight

The weight λ in Equation 3.5 was chosen on a subset of 3,000 MuSiQue training questions, sampled with 1,000 questions of each length, using the trained decomposer and the full pipeline. Table 4.10 reports the results. Performance forms a plateau between $\lambda = 0.4$ and 0.7: recall@1 is highest at 0.4 and 0.5, and chain@1, EM, and F1 are highest at 0.6, but all differences within the plateau are below 0.4 points. Pairwise McNemar tests on EM find no significant difference between 0.6 and any other value in the plateau ($p > 0.17$); only $\lambda = 0.3$ is noticeably lower ($p = 0.078$ against 0.6). We use $\lambda = 0.6$ as a value inside the plateau rather than

λ	Recall@1	Chain@1	EM	F1
0.3	0.758	0.525	0.464	0.549
0.4	**0.763**	0.533	0.467	0.552
0.5	**0.763**	0.536	0.468	0.554
0.6	0.762	**0.537**	**0.471**	**0.555**
0.7	0.761	0.535	0.470	0.554
0.8	0.759	0.531	0.470	0.552

Table 4.10: Results on 3,000 MuSiQue training questions for different values of λ (lenient chain@1).

as an optimum.

4.4.4 Final-Answer Prompt

The final-answer prompt asks the LLM for a short answer, such as an entity, a date, a number, or yes or no. An earlier version of the prompt consisted of this request alone. It implicitly treats the answer of the last hop as the answer to the question, which holds for the bridge questions of MuSiQue, a dataset that contains no comparison questions. On a comparison question, however, the answers of the hops are only the values being compared. For "Which film was released more recently, Royal Treasure or When Love Begins?", for example, the hops return two release dates, and the earlier prompt led the LLM to answer with one of the dates rather than with the title of the film. The current prompt adds a single instruction for such questions: the answers of the individual hops are the values used for the comparison, and the answer must name the entity that satisfies it. Both versions are given in Appendix B.

Table 4.11 reports EM with both prompts, for the control and for the method. Because the prompt affects only the final answer, the final answers are regenerated from the same chains, and retrieval and the answers of individual hops are unchanged. Under the control, the instruction raises EM by 0.99, 9.13, and 2.84 points on MuSiQue, 2WikiMultihopQA, and HotpotQA. The order follows the share of comparison questions: none in MuSiQue, 20% in HotpotQA, and 46% in 2WikiMultihopQA, counting comparison and bridge comparison questions. On 2WikiMultihopQA, the instruction therefore contributes more than the gate (8.02 points), whereas on MuSiQue and HotpotQA the gate contributes three to eleven

	Control ($\lambda = 0$)		Gate ($\lambda = 0.6$)	
Dataset	Without	With	Without	With
MuSiQue	0.319	0.329	0.429	**0.441**
2Wiki	0.430	0.521	0.501	**0.601**
HotpotQA	0.432	0.460	0.523	**0.552**

Table 4.11: EM with and without the instruction for comparison questions in the final-answer prompt. Only the final answer is regenerated; retrieval and the answers of individual hops are identical within each system.

times as much. The two contributions are nearly additive: the gain from the gate with the instruction exceeds the gain without it by at most 0.87 points.

4.4.5 Decomposer Training Data

The decomposer is trained on MuSiQue together with generated decompositions of 2WikiMultihopQA questions, which were sampled to include many comparison and bridge comparison questions. To measure the contribution of these data, we train a second decomposer on MuSiQue alone, with identical hyperparameters, and run the full method with it. On its own, the decomposer trained on MuSiQue alone reaches a slightly lower BLEU on MuSiQue (75.94 against 76.81) and predicts the correct number of hops for fewer four-hop questions (0.427 against 0.489). On 2WikiMultihopQA it never produces a four-hop decomposition, compared with 2,739 for the full decomposer.

Because the two decomposers predict different numbers of hops, the lenient metrics would favor the one that predicts fewer hops, as fewer hops are compared. Table 4.12 therefore reports strict chain@1 and recall@1 over all annotated hops. On 2WikiMultihopQA, removing the generated decompositions lowers recall@1 by 36.0 points, strict chain@1 by 30.0 points, and EM by 8.3 points. On HotpotQA, strict chain@1 falls by 5.1 points and EM by 0.8 points. On MuSiQue, the two decomposers are equal at the level of hops (-0.1 points recall@1), and the decomposer trained on MuSiQue alone is slightly better at the level of chains and answers ($+1.4$ points strict chain@1, $+0.7$ points EM).

The losses are concentrated on comparison questions, the type that MuSiQue does not contain. EM falls by 20.1 points on the bridge comparison questions

Dataset	Training data	Recall@1 (all hops)	Chain@1 (strict)	EM
MuSiQue	MuSiQue + 2Wiki	**0.660**	0.498	0.441
	MuSiQue only	0.659	**0.512**	**0.448**
2Wiki	MuSiQue + 2Wiki	**0.929**	**0.851**	**0.601**
	MuSiQue only	0.569	0.550	0.519
HotpotQA	MuSiQue + 2Wiki	**0.661**	**0.497**	**0.552**
	MuSiQue only	0.581	0.447	0.543

Table 4.12: The method with decomposers trained with and without the generated 2WikiMultihopQA decompositions. Recall@1 counts every annotated hop, with hops that are not predicted counted as misses.

and 12.6 points on the comparison questions of 2WikiMultihopQA, but only by 0.8 points on its compositional questions; on HotpotQA it falls by 4.1 points on comparison questions and 0.2 points on bridge questions. The generated decompositions therefore teach the decomposer to split comparison questions and long chains, at a small cost on MuSiQue.

4.5 Robustness

This section tests whether the gains hold when the retriever is replaced (Section 4.5.1) and when the passage pool of each question is replaced by a corpus shared by all questions (Section 4.5.2).

4.5.1 Alternative Retriever

We replace the BGE encoder with ColBERTv2 [40], a late-interaction retriever whose scores are not bounded. Its scores for the candidates of each hop are rescaled to $[0, 1]$ before Equation 3.5 is applied; the gate, the weight $\lambda = 0.6$, and the rest of the pipeline are unchanged, and the gate was not retrained. Table 4.13 shows that the gate improves ColBERT on all three datasets: recall@1 rises by 3.7 to 4.8 points, chain@1 by 7.0 to 9.3 points, and EM by 3.0 to 6.0 points. The gains are about half of those obtained with BGE, and ColBERT itself is weaker than BGE on all three datasets, which is why BGE is the default retriever.

Dataset	Retrieval	Recall@1	Recall@3	Chain@1	EM	F1
MuSiQue	ColBERTv2	0.618	0.758	0.373	0.332	0.416
	ColBERTv2 + gate	0.663	0.806	0.443	0.387	0.478
	BGE + gate	**0.700**	**0.844**	**0.513**	**0.441**	**0.536**
2Wiki	ColBERTv2	0.841	0.917	0.680	0.522	0.588
	ColBERTv2 + gate	0.878	0.964	0.754	0.553	0.623
	BGE + gate	**0.930**	**0.985**	**0.863**	**0.601**	**0.681**
HotpotQA	ColBERTv2	0.561	0.747	0.369	0.462	0.582
	ColBERTv2 + gate	0.608	0.819	0.462	0.522	0.650
	BGE + gate	**0.662**	**0.870**	**0.561**	**0.552**	**0.688**

Table 4.13: Results with ColBERTv2 as the retriever (lenient chain@1). The gate is the same as in the main experiments and is not retrained.

			Control ($\lambda = 0$)		Gate ($\lambda = 0.6$)	
Dataset	Metric	Distractor	$k = 10$	$k = 50$	$k = 10$	$k = 50$
MuSiQue	Recall@1	0.700	0.484	0.484	0.537	**0.547**
	Chain@1	0.513	0.229	0.229	0.298	**0.313**
	EM	0.441	0.232	0.232	0.298	**0.312**
2Wiki	Recall@1	0.930	0.743	0.743	0.785	**0.787**
	Chain@1	0.863	0.517	0.517	0.582	**0.586**
	EM	0.601	0.484	0.484	0.512	**0.518**
HotpotQA	Recall@1	0.662	0.479	0.479	0.533	**0.552**
	Chain@1	0.561	0.289	0.289	0.373	**0.405**
	EM	0.552	0.375	0.375	0.436	**0.459**

Table 4.14: Open-corpus results with k candidates per hop (lenient chain@1). The distractor column repeats the main result of the method for reference.

4.5.2 Open-Corpus Retrieval

In the open-corpus setting, the passages of all development questions of a dataset are merged into one corpus, with duplicates removed, giving 21,100 passages for MuSiQue, 56,687 for 2WikiMultihopQA, and 66,635 for HotpotQA. Every hop retrieves from this corpus instead of the pool of its own question. We run both the control and the method with $k = 10$ and $k = 50$ candidates per hop. Table 4.14 reports the results.

The gate remains effective in the open corpus. Relative to the control, it raises recall@1 by 4.2 to 7.3 points, chain@1 by 6.4 to 11.6 points, and EM by 2.8 to 8.4

points, depending on the dataset and k; the gains in strict chain@1 are of the same size. These gains are about half of those in the distractor setting.

Enlarging the candidate set from 10 to 50 passages per hop helps only together with the gate. Under the control, recall@1 is unchanged on MuSiQue and HotpotQA and differs by 0.01 points on 2WikiMultihopQA, since the passage with the highest retrieval score does not depend on how many candidates are kept. With the gate, it rises by 1.0, 0.2, and 2.0 points, because the gate can move an annotated passage ranked between 11 and 50 to the first position, which it does for 24%, 46%, and 37% of the hops whose annotated passage falls in that range.

Even so, the open corpus remains far harder than the distractor setting: with $k = 50$, recall@1 is 11 to 15 points lower. The limiting factor is the first retrieval stage. For 22.6%, 13.4%, and 29.0% of the hops, the annotated passage is not among the 50 passages with the highest retrieval score, so no reranking of the candidates can select it.

4.6 Comparison with Prior Systems

We compare the method with two systems reviewed in Section 2.1, ChainRAG [19] and GRITHopper [12]. ChainRAG decomposes the question and expands the evidence of each sub-question through a sentence graph, without training. GRITHopper fine-tunes a 7B decoder model to retrieve evidence iteratively without decomposition. We run both systems with their released code on the same development sets.

Protocol. Neither system aligns its retrieval steps with the annotated hops, so we use order-invariant metrics: a selected passage counts as correct if it is any annotated supporting passage of the question. Order-invariant recall@1 and recall@3 are computed per retrieval step, coverage is the fraction of annotated passages contained in the evidence a system finally uses, and full coverage is the fraction of questions for which this evidence contains all of them. Because the two systems record only the titles of their passages, passages are matched by title for all three systems. Matching by title overestimates recall when a pool contains several passages with the same title; for the method, it raises order-invariant recall@1 on MuSiQue from 0.856 to

		Recall (order-invariant)		Coverage			
Dataset	System	@1	@3	Mean	Full	EM	F1
MuSiQue	ChainRAG	0.672	0.863	**0.959**	**0.905**	0.276	0.385
	GRITHopper	**0.906**	**0.986**	0.821	0.609	0.279	0.373
	Ours	0.883	0.974	0.797	0.565	**0.441**	**0.536**
2Wiki	ChainRAG	0.920	0.979	**0.987**	**0.970**	0.443	0.522
	GRITHopper	0.950	0.996	0.892	0.775	0.539	0.627
	Ours	**0.973**	**0.997**	0.939	0.873	**0.601**	**0.681**
HotpotQA	ChainRAG	0.803	0.950	**0.986**	**0.974**	0.460	0.603
	GRITHopper	**0.966**	**0.996**	0.946	0.907	**0.555**	**0.693**
	Ours	0.899	0.981	0.853	0.731	0.552	0.688

Table 4.15: Comparison with prior systems. Recall is order-invariant and passages are matched by title. Coverage is the fraction of annotated passages in the evidence finally used (mean over questions) and the fraction of questions for which all of them are included (full).

0.883, and all three systems should be read as upper bounds. For the answers, all three systems use Llama-3.1-8B-Instruct as the generator. ChainRAG generates its answers with its own procedure. GRITHopper has no answer-generation step, so we added a reader that receives the question and the retrieved passages, and we deliberately gave it the same instruction for comparison questions that our final-answer prompt contains.

Results. Table 4.15 reports the results. In retrieval, GRITHopper is the strongest system on MuSiQue and HotpotQA and the method on 2WikiMultihopQA. ChainRAG has the lowest recall but the highest coverage, because it passes many passages to the final answer. In answer quality, the method is best on MuSiQue and 2WikiMultihopQA, ahead of the better of the two other systems by 16.2 and 6.2 points EM, and comparable to GRITHopper on HotpotQA (0.552 against 0.555 EM).

The coverage of ChainRAG comes at the cost of precision. Table 4.16 reports the proportion of the passages finally used that are annotated supporting passages. ChainRAG uses 14.5, 7.6, and 7.9 passages per question on average, of which only 21%, 36%, and 28% are supporting; the method and GRITHopper use about two passages per question, roughly the number of annotated ones, with a precision of 0.88 to 0.97. The larger and noisier context of ChainRAG is consistent with its

Dataset	System	Passages used	Precision	Coverage
MuSiQue	ChainRAG	14.54	0.207	**0.959**
	GRITHopper	2.31	**0.909**	0.821
	Ours	2.29	0.882	0.797
2Wiki	ChainRAG	7.61	0.356	**0.987**
	GRITHopper	2.24	0.954	0.892
	Ours	2.37	**0.968**	0.939
HotpotQA	ChainRAG	7.86	0.275	**0.986**
	GRITHopper	1.95	**0.966**	0.946
	Ours	1.91	0.896	0.853

Table 4.16: Average number of passages in the evidence finally used for answering, and the proportion of them that are annotated supporting passages (precision). The questions have 2.60, 2.44, and 2.00 supporting passages on average.

lower answer quality despite its high coverage.

Two differences in the setup affect the answer columns. First, the readers differ: GRITHopper receives the same instruction for comparison questions as the method, but ChainRAG does not. Without this instruction, the method still reaches 0.429, 0.501, and 0.523 EM (Table 4.11), ahead of ChainRAG by 15.2, 5.9, and 6.3 points, so its advantage over ChainRAG does not depend on the instruction. Second, the systems differ in what they train. ChainRAG requires no training, GRITHopper fine-tunes a 7B model for multi-hop retrieval, and the method trains a decomposer and a linear gate while leaving the retriever and the generator unchanged.

Chapter 5

Discussion

Chapter plan

- Using traces with multiple errors, show that the gate detects whether the prefix has been contaminated rather than which hop is wrong, and that its signal comes mainly from the accumulated state.
- This behavior is consistent with gains concentrating at the chain level, and with later correct candidates being penalized after a first mistake (write as "consistent with", not as a causal explanation).
- Remaining failure modes of the full system.
- Limitations and future directions.

[**TODO:** Roadmap paragraph: "This chapter X. Section N.1 .... Section N.2 ...."]

5.1 What the Gate Detects

[**TODO:** Multi-error traces: scores rise after the first error and stay high; signal comes from $\mathbf{h}$, not $\mathbf{\Delta}$; a prefix-contamination detector.]

5.2 Error Analysis

[**TODO:** NA cascade, reader picks the wrong hop, EM strictness, shortened chains, retrieval gains not reaching answers.]

5.3 Limitations

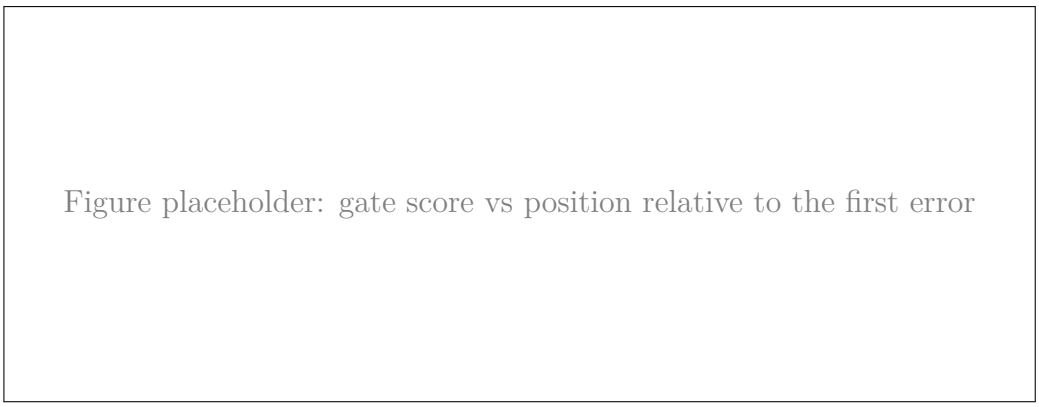


Figure 5.1: Gate score relative to the first error in the chain.

[**TODO:** Run-in bold paragraphs, e.g. **Per-hop supervision.** ...; each says what the limitation is, why it exists, what would remove it. Topics: per-hop gold labels (MuSiQue only); error amplification; no backtracking; recall bottleneck in open corpus; inference overhead (one prefill per candidate).]

5.4 Future Directions

[**TODO:** One paragraph per direction, each tied to a limitation: path-level scoring for beam pruning; steering; better first-stage recall.]

Chapter 6

Conclusion

Chapter plan

- Restate the problem and the approach in one paragraph.
- Summarise the main results and the mechanism finding.
- Close with the broader takeaway: hidden states are a cheap, reusable signal for guiding retrieval.

[**TODO:** Three paragraphs, no sections: (1) what was investigated and how; (2) main results with key numbers; (3) limitations, then the significance and the foundation it lays.]

Bibliography

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks", in *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474.

[2] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering", in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 2369–2380.

[3] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps", in *Proceedings of the 28th International Conference on Computational Linguistics*, Barcelona, Spain (Online): International Committee on Computational Linguistics, 2020, pp. 6609–6625.

[4] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop questions via single-hop question composition", *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022.

[5] S. Min, V. Zhong, L. Zettlemoyer, and H. Hajishirzi, "Multi-hop reading comprehension through question decomposition and rescoring", in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Florence, Italy: Association for Computational Linguistics, 2019, pp. 6097–6109.

[6] O. Press, M. Zhang, S. Min, L. Schmidt, N. Smith, and M. Lewis, "Measuring and narrowing the compositionality gap in language models", in *Findings*

of the Association for Computational Linguistics: EMNLP 2023, Singapore: Association for Computational Linguistics, 2023, pp. 5687–5711.

[7] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions", in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Toronto, Canada: Association for Computational Linguistics, 2023, pp. 10 014–10 037.

[8] Z. Jiang, F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation", in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore: Association for Computational Linguistics, 2023, pp. 7969–7992.

[9] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection", in *International Conference on Learning Representations*, 2024, pp. 9112–9141.

[10] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity", in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Mexico City, Mexico: Association for Computational Linguistics, 2024, pp. 7036–7050.

[11] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation", 2024. arXiv: `2401.15884 [cs.CL]`.

[12] J.-J. Erker, N. Reimers, and I. Gurevych, "GRITHopper: Decomposition-free multi-hop dense retrieval", in *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, Rabat, Morocco: Association for Computational Linguistics, 2026, pp. 77–94.

[13] L. Sun, H. Dong, B. Qiao, Q. Lin, D. Zhang, and S. Rajmohan, "LLM reasoning as trajectories: Step-specific representation geometry and correctness signals", 2026. arXiv: `2604.05655 [cs.CL]`.

[14] S. Min, E. Wallace, S. Singh, M. Gardner, H. Hajishirzi, and L. Zettlemoyer, "Compositional questions do not necessitate multi-hop reasoning", in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Florence, Italy: Association for Computational Linguistics, 2019, pp. 4249–4257.

[15] W. Xiong, X. L. Li, S. Iyer, J. Du, P. Lewis, W. Y. Wang, Y. Mehdad, W.-t. Yih, S. Riedel, D. Kiela, and B. Ouz, "Answering complex open-domain questions with multi-hop dense retrieval", in *International Conference on Learning Representations*, 2021. [Online]. Available: `https://openreview.net/forum?id=EMHoBG0avc1`

[16] O. Khattab, C. Potts, and M. Zaharia, "Baleen: Robust multi-hop reasoning at scale via condensed retrieval", in *Advances in Neural Information Processing Systems*, vol. 34, Curran Associates, Inc., 2021, pp. 27 670–27 682.

[17] J. Zhang, H. Zhang, D. Zhang, L. Yong, and S. Huang, "End-to-end beam retrieval for multi-hop question answering", in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Mexico City, Mexico: Association for Computational Linguistics, 2024, pp. 1718–1731.

[18] T. Wolfson, M. Geva, A. Gupta, M. Gardner, Y. Goldberg, D. Deutch, and J. Berant, "Break it down: A question understanding benchmark", *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 183–198, 2020.

[19] R. Zhu, X. Liu, Z. Sun, Y. Wang, and W. Hu, "Mitigating lost-in-retrieval problems in retrieval augmented multi-hop question answering", in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vienna, Austria: Association for Computational Linguistics, 2025, pp. 22 362–22 375.

[20] W. Su, Y. Tang, Q. Ai, Z. Wu, and Y. Liu, "DRAGIN: Dynamic retrieval augmented generation based on the real-time information needs of large language models", in *Proceedings of the 62nd Annual Meeting of the Association*

for Computational Linguistics (Volume 1: Long Papers), Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 12 991–13 013.

[21] Y. Wang, R. Ren, J. Li, W. X. Zhao, J. Liu, and J.-R. Wen, "REAR: A relevance-aware retrieval-augmented framework for open-domain question answering", in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Miami, Florida, USA: Association for Computational Linguistics, 2024, pp. 5613–5626.

[22] A. Gupta, S. Sundaresan, and D. Mahapatra, "RSRank: Learning relevance from representational shifts", 2026. arXiv: `2606.17468 [cs.IR]`.

[23] S. M. Huq, C. Child, T. Weyde, and P. Madhyastha, "Knowing before answering: Decoding language models for reliable RAG", 2026. arXiv: `2608.27661 [cs.CL]`.

[24] Z. Yao, W. Qi, L. Pan, S. Cao, L. Hu, L. Weichuan, L. Hou, and J. Li, "SeaKR: Self-aware knowledge retrieval for adaptive retrieval augmented generation", in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vienna, Austria: Association for Computational Linguistics, 2025, pp. 27 022–27 043.

[25] G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes", 2016. arXiv: `1610.01644 [stat.ML]`.

[26] Y. Belinkov, "Probing classifiers: Promises, shortcomings, and advances", *Computational Linguistics*, vol. 48, no. 1, pp. 207–219, 2022.

[27] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez, N. Schiefer, Z. Hatfield-Dodds, N. DasSarma, E. Tran-Johnson, S. Johnston, S. El-Showk, A. Jones, N. Elhage, T. Hume, A. Chen, Y. Bai, S. Bowman, S. Fort, D. Ganguli, D. Hernandez, J. Jacobson, J. Kernion, S. Kravec, L. Lovitt, K. Ndousse, C. Olsson, S. Ringer, D. Amodei, T. Brown, J. Clark, N. Joseph, B. Mann, S. McCandlish, C. Olah, and J. Kaplan, "Language models (mostly) know what they know", 2022. arXiv: `2207.05221 [cs.CL]`.

[28] A. Azaria and T. Mitchell, "The internal state of an LLM knows when it's lying", in *Findings of the Association for Computational Linguistics: EMNLP*

2023, Singapore: Association for Computational Linguistics, 2023, pp. 967–976.

[29] C. Burns, H. Ye, D. Klein, and J. Steinhardt, "Discovering latent knowledge in language models without supervision", in *International Conference on Learning Representations*, 2023.

[30] S. Marks and M. Tegmark, "The geometry of truth: Emergent linear structure in large language model representations of true/false datasets", in *First Conference on Language Modeling*, 2024.

[31] C. Chen, K. Liu, Z. Chen, Y. Gu, Y. Wu, M. Tao, Z. Fu, and J. Ye, "INSIDE: LLMs' internal states retain the power of hallucination detection", in *International Conference on Learning Representations*, 2024, pp. 3056–3076.

[32] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg, "Inference-time intervention: Eliciting truthful answers from a language model", in *Advances in Neural Information Processing Systems*, vol. 36, Curran Associates, Inc., 2023, pp. 41 451–41 530.

[33] A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, X. Yin, M. Mazeika, A.-K. Dombrowski, S. Goel, N. Li, M. J. Byun, Z. Wang, A. Mallen, S. Basart, S. Koyejo, D. Song, M. Fredrikson, J. Z. Kolter, and D. Hendrycks, "Representation engineering: A top-down approach to AI transparency", 2023. arXiv: `2310.01405 [cs.LG]`.

[34] W. Kim, Y. In, S. Park, D. Lee, and C. Park, "PAIR: Prefix-aware internal reward model for multi-turn agent optimization", 2026. arXiv: `2605.17877 [cs.AI]`.

[35] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings", in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, Association for Computing Machinery, 2024.

[36] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, et al., "The Llama 3 herd of models", 2024. arXiv: `2407.21783 [cs.AI]`.

[37] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension", in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Online: Association for Computational Linguistics, 2020, pp. 7871–7880.

[38] L. van der Maaten and G. Hinton, "Visualizing data using t-sne", *Journal of Machine Learning Research*, vol. 9, no. 86, pp. 2579–2605, 2008.

[39] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: A method for automatic evaluation of machine translation", in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, P. Isabelle, E. Charniak, and D. Lin, Eds., Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, Jul. 2002, pp. 311–318. [Online]. Available: https://aclanthology.org/P02-1040/

[40] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, "ColBERTv2: Effective and efficient retrieval via lightweight late interaction", in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, M. Carpuat, M.-C. de Marneffe, and I. V. Meza Ruiz, Eds., Seattle, United States: Association for Computational Linguistics, Jul. 2022, pp. 3715–3734. [Online]. Available: https://aclanthology.org/2022.naacl-main.272/

Table placeholder: explored designs and outcomes

Table A.1: Design explorations that were not adopted.

Appendix A

Design Explorations

[**TODO:** One table of explored-but-not-adopted designs (motivation, result, reason), with short notes on late fusion, context gate, decoupled placeholders and adaptive λ.]

Appendix B

Prompts

This appendix gives the prompts with which the LLM generates the answer of each hop and the final answer (Section 3.2.2). Text in angle brackets, such as ⟨*question*⟩, is filled in for each question. Every prompt is rendered with the chat template of Llama-3.1-8B-Instruct, and answers are generated with greedy decoding, with at most 64 new tokens for the answer of a hop and 128 for the final answer.

B.1 Answer of Each Hop

The prompt for the answer of hop j is a single user turn without a system prompt. The earlier steps list, for each completed hop, the expanded sub-question, the text of the selected passage without its title, and the answer of the hop; at the first hop, this part is omitted. The selected passage of hop j is given with its title.

Original question:
⟨*question*⟩
Previous reasoning steps:
Step 1:
Subquestion: ⟨*expanded sub-question of hop 1*⟩
Selected evidence: ⟨*text of the passage selected at hop 1*⟩
Answer: ⟨*answer of hop 1*⟩
...
Current subquestion:
⟨*expanded sub-question of hop j*⟩
Selected evidence title: ⟨*title of the selected passage*⟩
Selected evidence:
⟨*text of the selected passage*⟩
Read the selected evidence and answer the current subquestion with a SHORT span or phrase only (no full sentences or explanation). If numeric, output the number. If the passage does not contain the answer, output exactly: NA
Answer:

Only the first line of the output is kept, and a trailing period is removed. An empty output becomes NA, as does an output longer than 40 characters that contains one of the phrases “no information”, “not mentioned”, “not specified”, “cannot be determined”, “does not contain”, “doesn’t contain”, “not found in the passage”, or “unknown”. The resulting answer fills the placeholders [Answer j] of later subquestions.

B.2 Final Answer

The final answer is generated from a system prompt and a user turn. The system prompt is:

You are a precise multi-hop question answering reader. Use only the provided reasoning trace. After re-reading the original question, give a direct short answer on exactly one line: Final answer: <short answer>. Do not write anything after the final answer line.

The user turn lists every hop of the chain, followed by the question and the instructions. The current prompt contains the instruction for comparison questions marked below; the earlier prompt examined in Section 4.4.4 is identical except that

this instruction is absent.

Reasoning trace:

Step 1:

Subquestion: ⟨*expanded sub-question of hop 1*⟩

Selected evidence: ⟨*text of the passage selected at hop 1*⟩

Answer: ⟨*answer of hop 1*⟩

...

Step K:

Subquestion: ⟨*expanded sub-question of hop K*⟩

Selected evidence: ⟨*text of the passage selected at hop K*⟩

Answer: ⟨*answer of hop K*⟩

Re-read the original question before answering:

⟨*question*⟩

Answer the original question directly using only the reasoning trace above.

Give a short span or phrase only (entity, date, number, or yes/no).

⟨*instruction for comparison questions; current prompt only*⟩

For yes/no questions, answer yes or no only.

Output exactly one line:

Final answer: <short answer>

The instruction for comparison questions is:

If the original question asks you to COMPARE two or more entities (for example "which was released more recently", "who was born first", "which is longer"), the per-step answers above are the values used to make that comparison, not the final answer themselves -- you must perform the comparison yourself and answer with the NAME of the entity that satisfies it, not a date, number, or other raw value.

The final answer is taken from the last line of the output that begins with "Final answer:" (case-insensitive in "answer", optionally preceded by "###"); quotation marks, Markdown emphasis characters, and a trailing period are removed. If no such line exists, the answer of the last hop is used instead.